

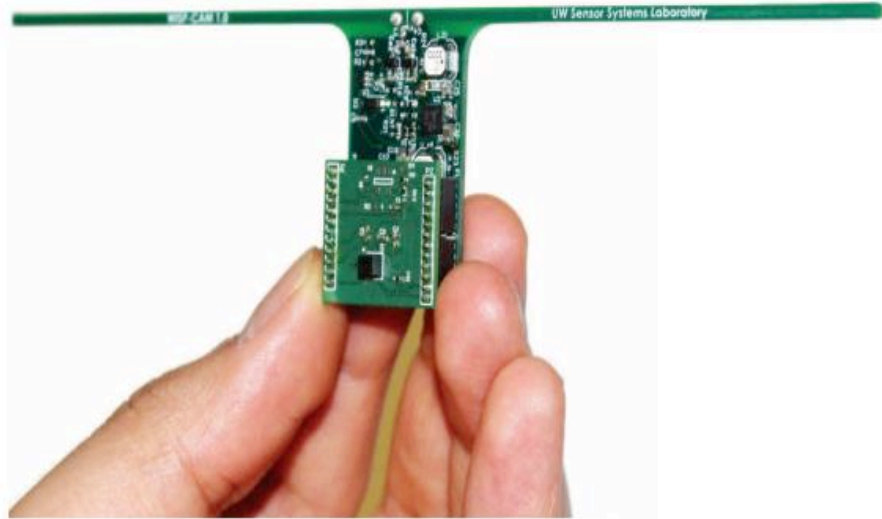
EE 186: Introduction to Embedded Systems

Lecture 3

Zerina Kapetanovic

Some *Interesting* Embedded Systems

These are both "battery-free" devices



What are we learning about today?

- Deeper Dive: Stack and Application Binary Interface (ABI)
- Memory-mapped I/O (MMIO)
- Bus Architectures
- Advanced Microcontroller Bus Architecture (AMBA)

Optional Readings

- Embedded Systems w/ ARM Cortex-M Microcontrollers in Assembly Language and C (Yifeng Zhu)
 - Chapter 8.1, 8.2, 8.6
 - Chapter 14.1, 14.2, 14.3, 14.6
 - Chapter 19.1

Application Binary Interface

A “contract” between programmers on how to share hardware resources

When you call a function written by another programmer, that function will need to use registers.

What happens if you have important data in those registers?

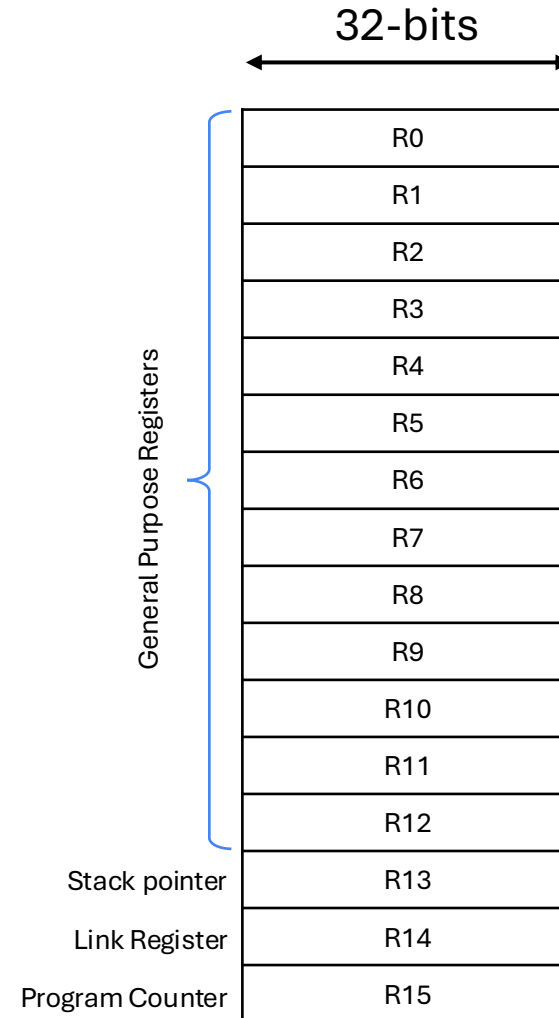
Application Binary Interface (ABI)

Some Basic Rules:

1. A subroutine must preserve the contents of the registers R4-R11 and SP
2. Arguments are passed through R0 to R3
3. Return value is placed in R0
4. Allocate space in the stack as needed. Use it as needed.
5. Push and pop an even number of registers to maintain an 8-byte alignment in the stack.

Processor Registers

- Registers are high-speed storage inside of the processor
- In this case, the register stores **32-bit values**
- **R0-R12** are general-purpose registers
- **R13:** Stack Pointer (SP)
- **R14:** Link Register (LR)
- **R15:** Program Counter (PC)



Violation Example

Caller Program

```
MOV r4, #100
...
BL foo
...
ADD r4, r4, #1      ; r4 = 101, not 11
```

Subroutine/Callee

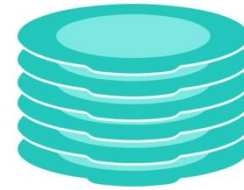
```
foo PROC
...
MOV r4, #10        ; foo changes r4
...
BX LR
ENDP
```

How can we solve this issue?

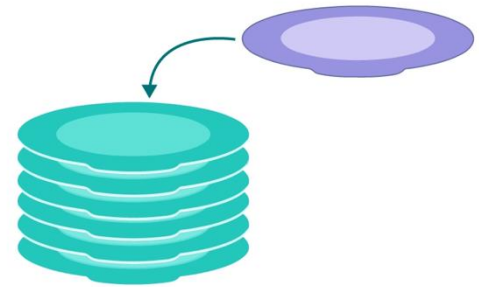
Stack

- The **stack** is last-in-first-out temporary storage
- On the ARM Cortex – M processor, the **stack always operates on 32-bit data**
- The **stack pointer (SP)** always points to the 32-bit data on top of the stack

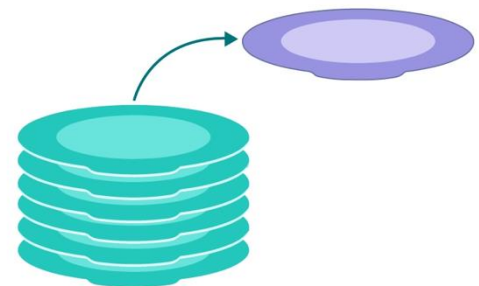
Stack of plates



Adding a plate to the stack



Removing a plate from the stack



Stack

- ARM uses a **full descending** stack
- The stack **grows downward in memory** as we push data on it
- A **full stack** is if the memory location pointed to by the SP holds the last item that was pushed onto the stack
- **PUSH:** adds an item to the top of the stack
- **POP:** removes the item that was added last

PUSH {Rd}

- The stack grows downwards so to PUSH we first decrement SP by 4
 - $SP = SP - 4$
 - $(*SP) = Rd$

Push Multiple Registers

`PUSH {r7, r6, r8}`  `PUSH {r8, r7, r6}`  `PUSH {r8}`
`PUSH {r7}`
`PUSH {r6}`

POP {Rd}

- POP is the opposite
 - $Rd = (*SP)$
 - $SP = SP + 4$

Pop Multiple Registers

`POP {r7, r6, r8}`  `POP {r8, r7, r6}`  `POP {r6}`
`POP {r7}`
`POP {r8}`

Stack Example

Assume that initially R0 contains the value 1, R1 contains the value 2, and R2 contains the value 3

```
PUSH {R0}
```

```
PUSH {R1}
```

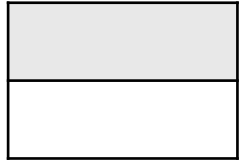
```
PUSH {R2}
```

```
POP {R3}
```

```
POP {R4}
```

```
POP {R5}
```

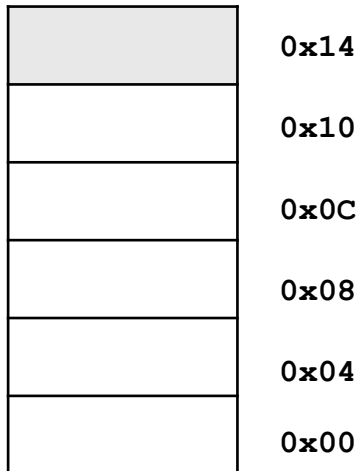
Stack Example



= data stored on the stack

= locations in the stack area but do not contain stack data

SP →



PUSH {R0}

PUSH {R1}

PUSH {R2}

POP {R3}

POP {R4}

POP {R5}

Assume that initially

R0 = 1

R1 = 2

R2 = 3

Stack Example

PUSH {R0}

PUSH {R1}

PUSH {R2}

POP {R3}

POP {R4}

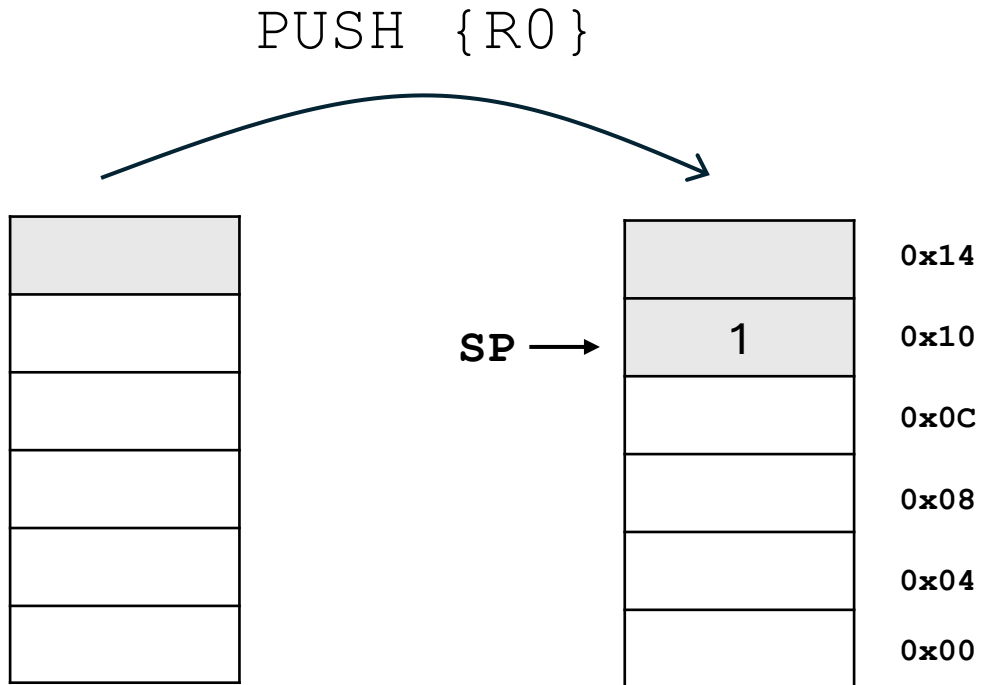
POP {R5}

Assume that initially

R0 = 1

R1 = 2

R2 = 3



Stack Example

PUSH {R0}

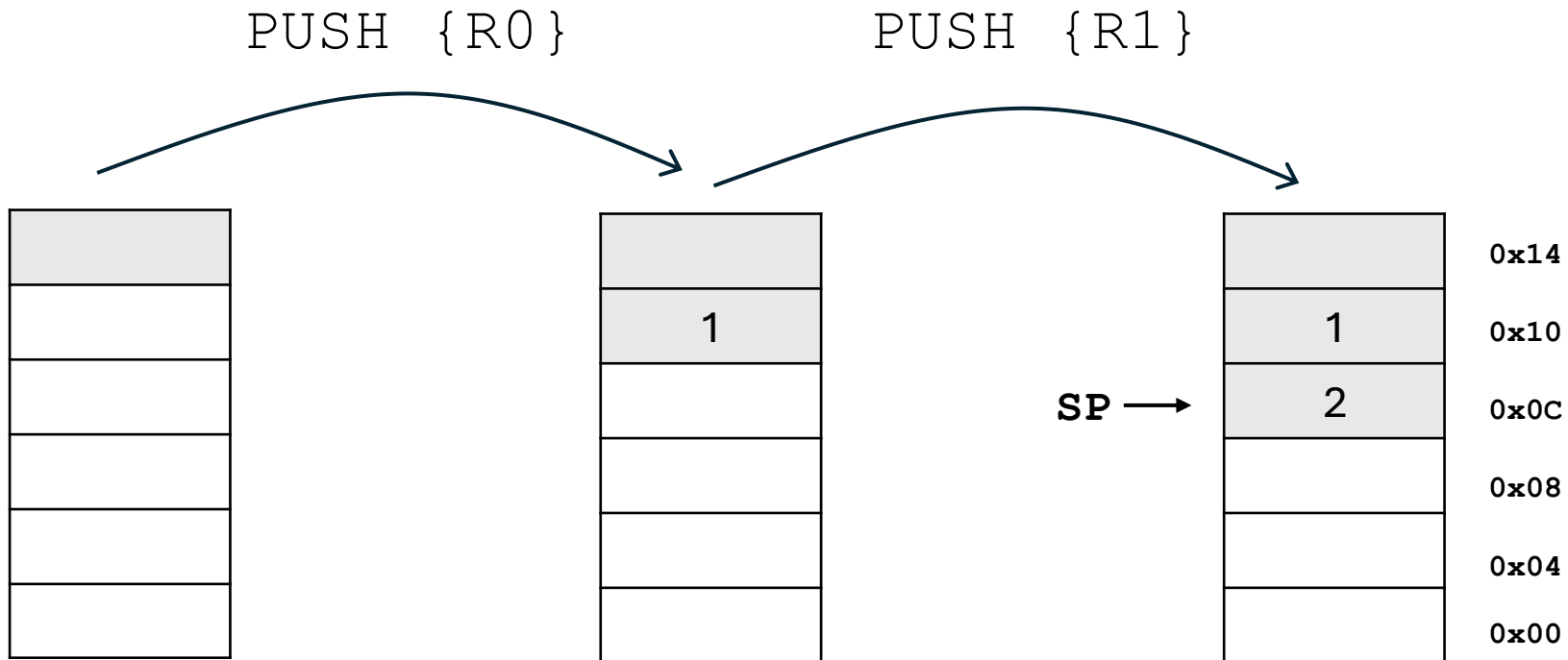
PUSH {R1} Assume

PUSH {R2} R0 = 1

POP {R3} R1 = 2

POP {R4} R2 = 3

POP {R5}



Stack Example

PUSH {R0}

PUSH {R1}

PUSH {R2}

POP {R3}

POP {R4}

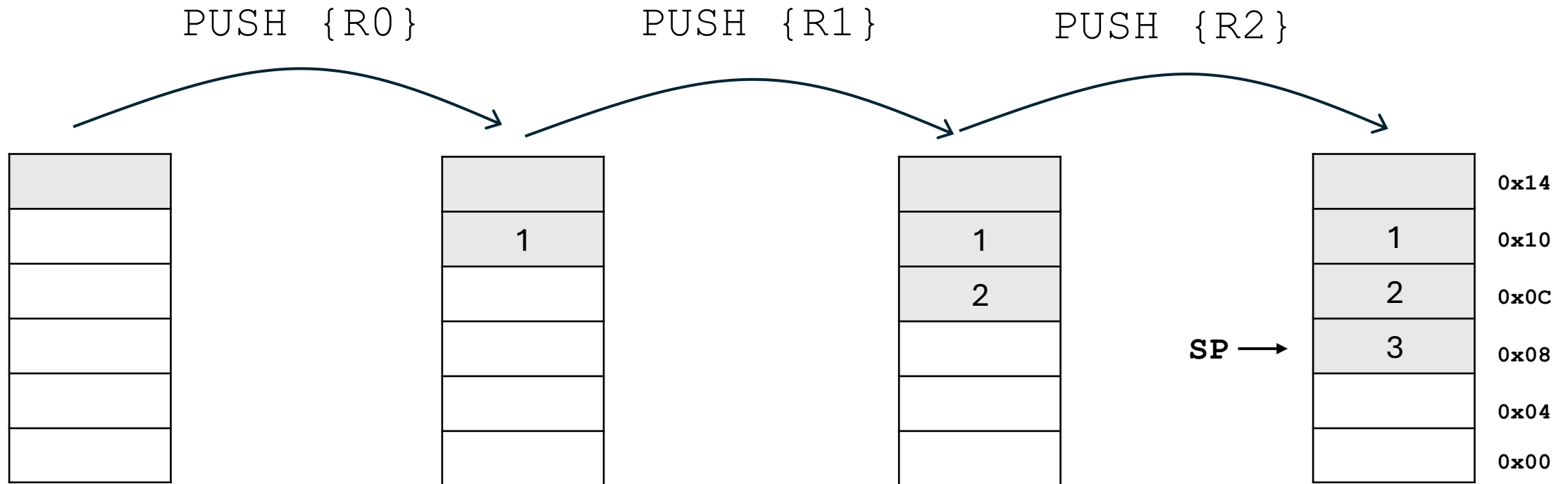
POP {R5}

Assume

R0 = 1

R1 = 2

R2 = 3



Stack Example

PUSH {R0}

PUSH {R1}

PUSH {R2}

POP {R3}

POP {R4}

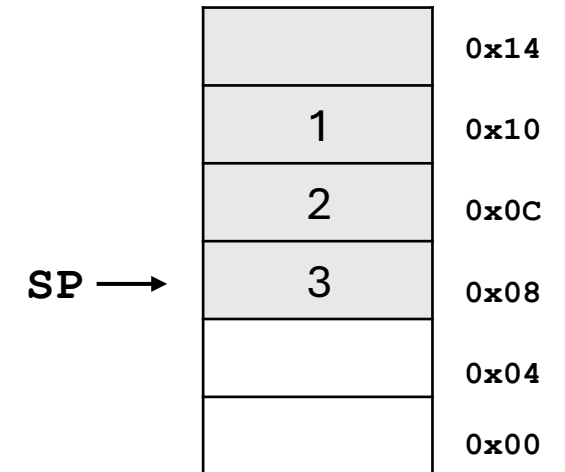
POP {R5}

Assume

R0 = 1

R1 = 2

R2 = 3



Stack Example

PUSH {R0}

PUSH {R1}

PUSH {R2}

POP {R3}

POP {R4}

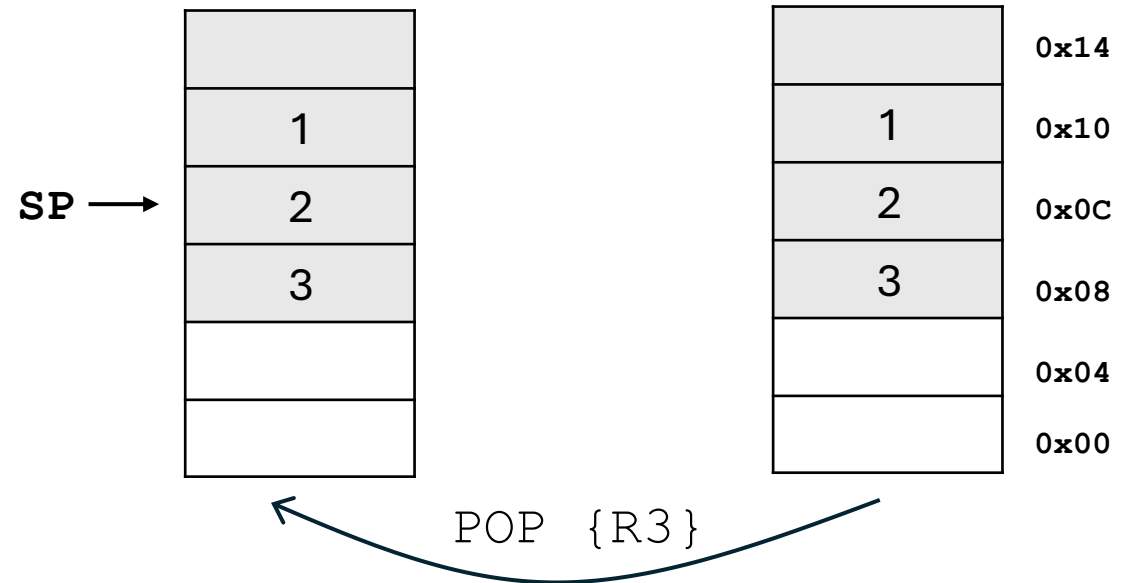
POP {R5}

Assume

R0 = 1

R1 = 2

R2 = 3



Stack Example

PUSH {R0}

PUSH {R1}

PUSH {R2}

POP {R3}

POP {R4}

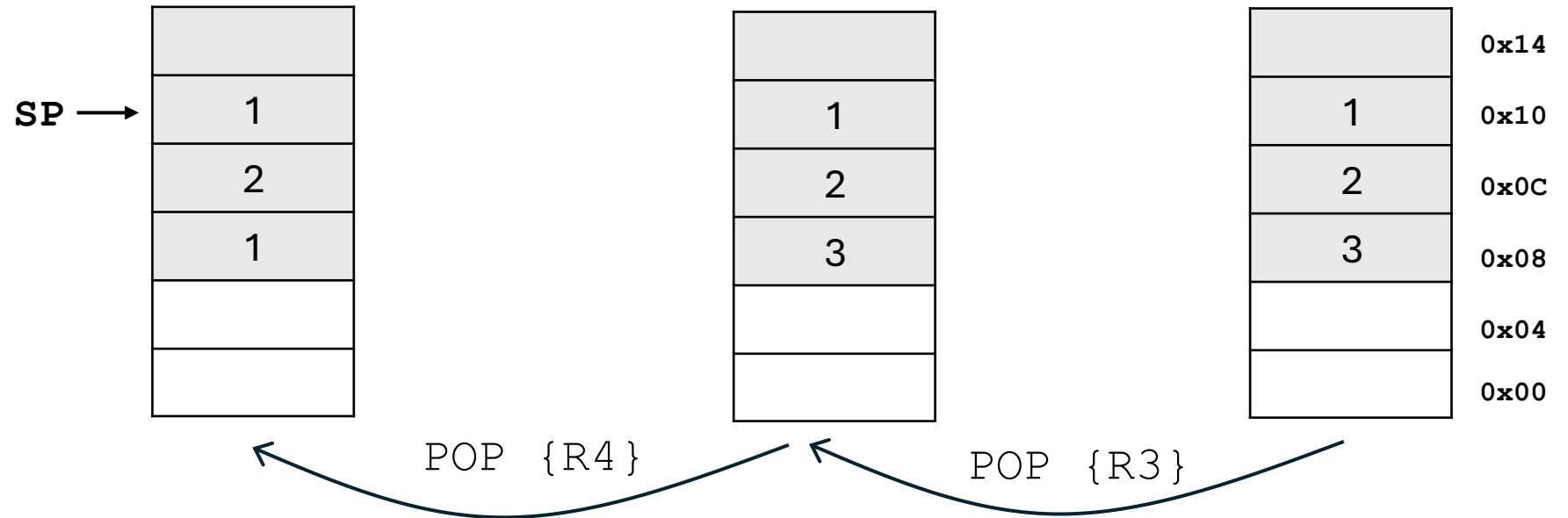
POP {R5}

Assume

R0 = 1

R1 = 2

R2 = 3



Stack Example

PUSH {R0}

PUSH {R1}

PUSH {R2}

POP {R3}

POP {R4}

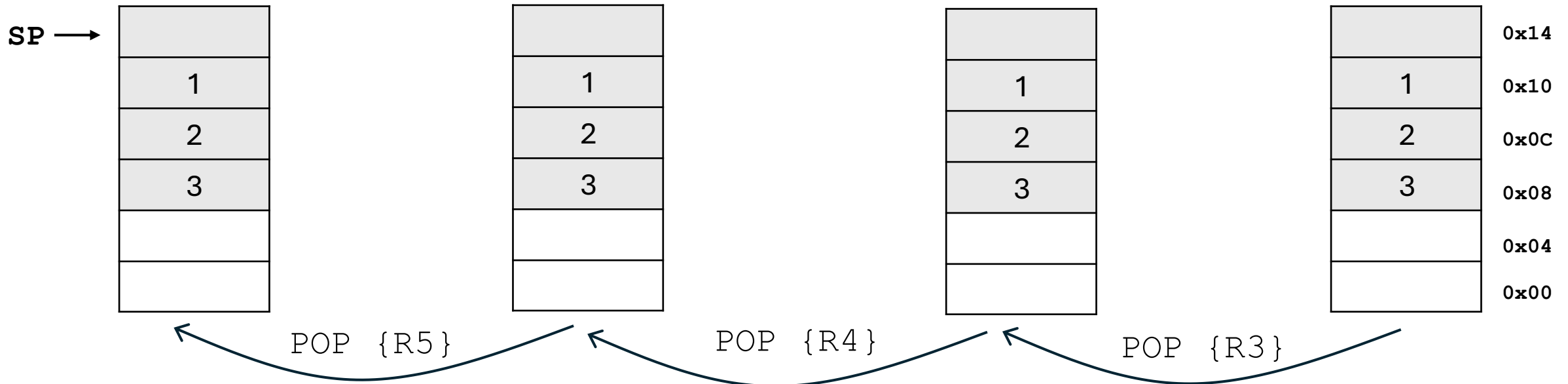
POP {R5}

Assume

R0 = 1

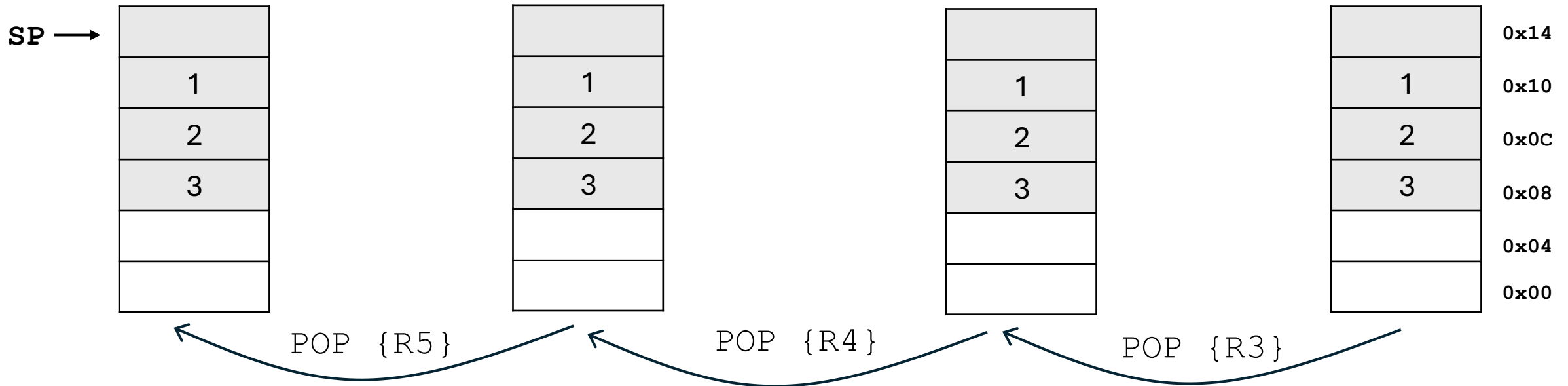
R1 = 2

R2 = 3



Stack Example

- We pushed contents from registers R0, R1, and R2 onto the stack
- We popped values off of the stack and placed them into registers R3, R4, and R5
- R3 = 3, R4 = 2, and R5 = 1



Preserve the runtime environment via Stack

Caller Program

```
MOV r4, #100
...
BL foo
...
ADD r4, r4, #1      ; r4 = 101
```

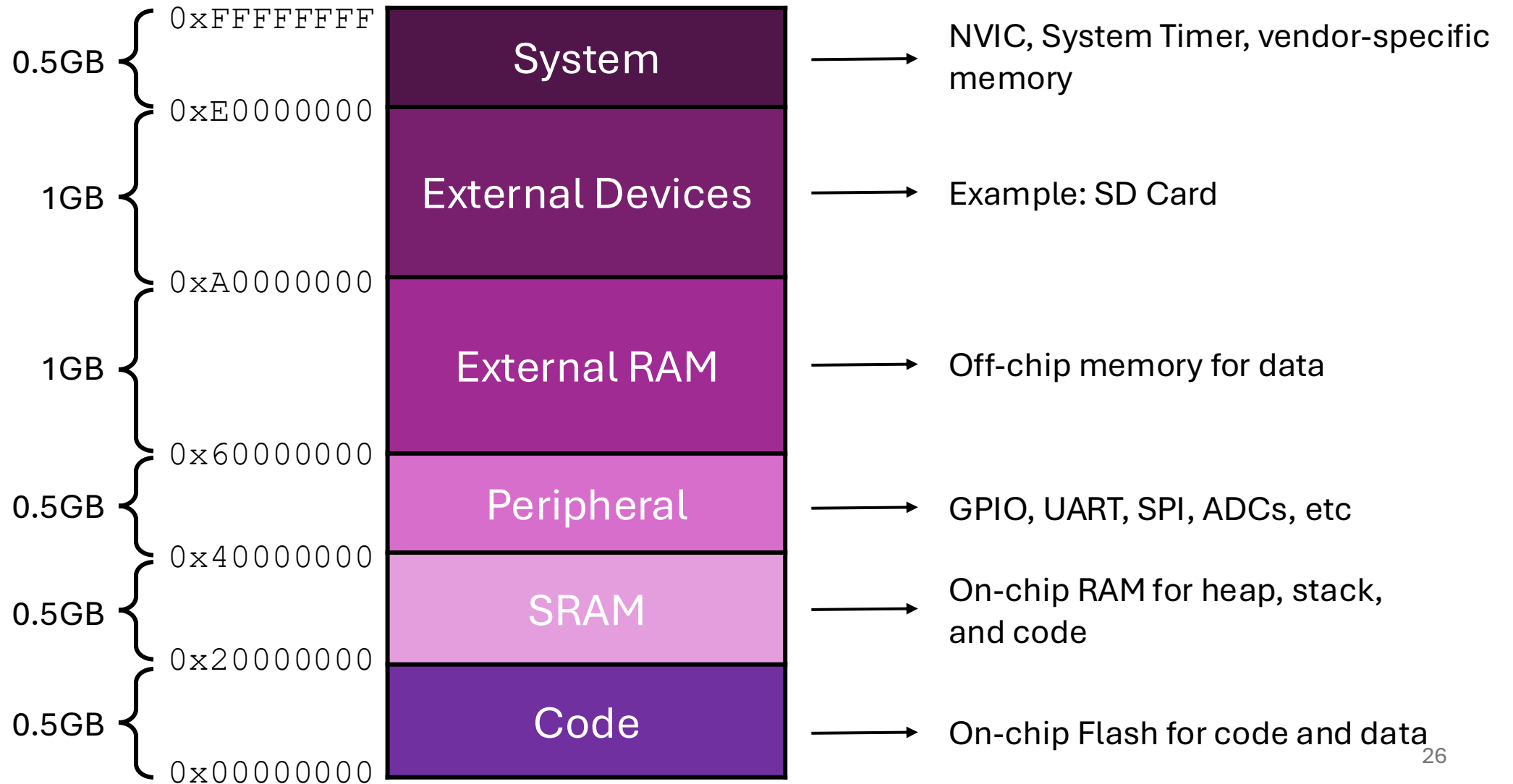
Subroutine/Callee

```
foo PROC
    PUSH {r4}      ; preserve r4
    ...
    MOV r4, #10    ; foo changes r4
    ...
    POP {r4}       ; recover r4
    BX LR
ENDP
```

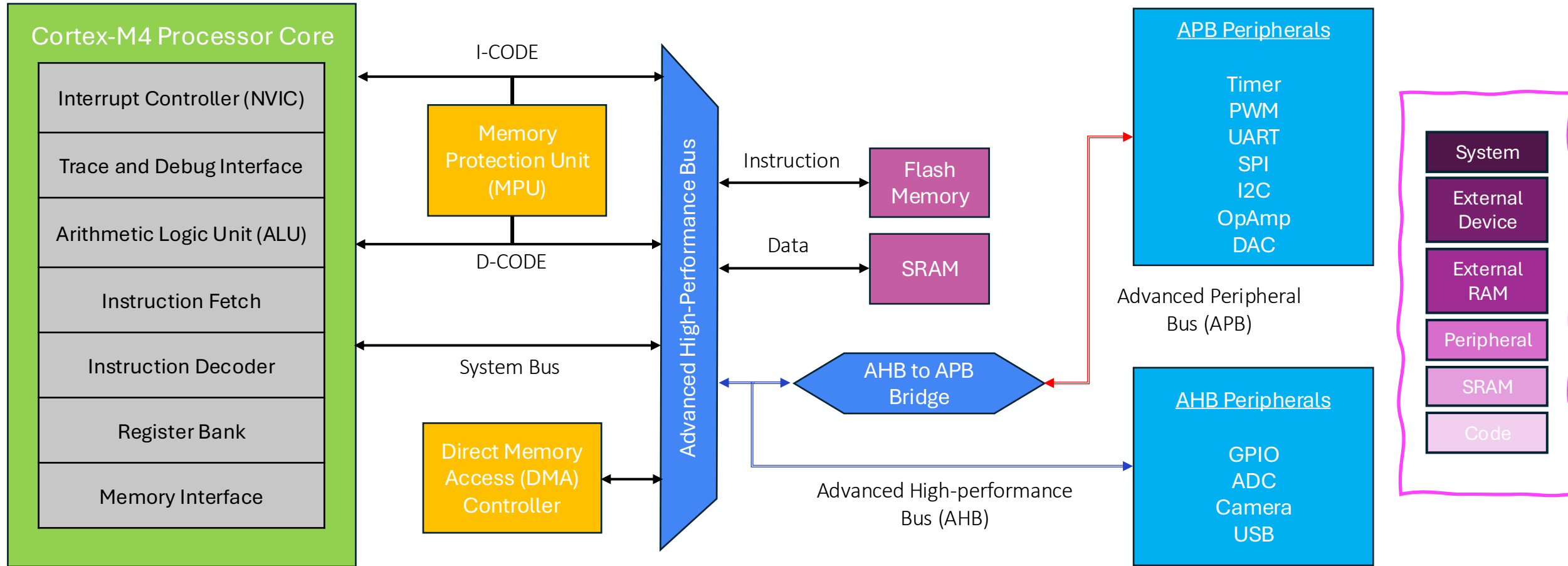
What are we learning about today?

- Deeper Dive: Stack and Application Binary Interface (ABI)
- **Memory-mapped I/O (MMIO)**
- Bus Architectures
- Advanced Microcontroller Bus Architecture (AMBA)

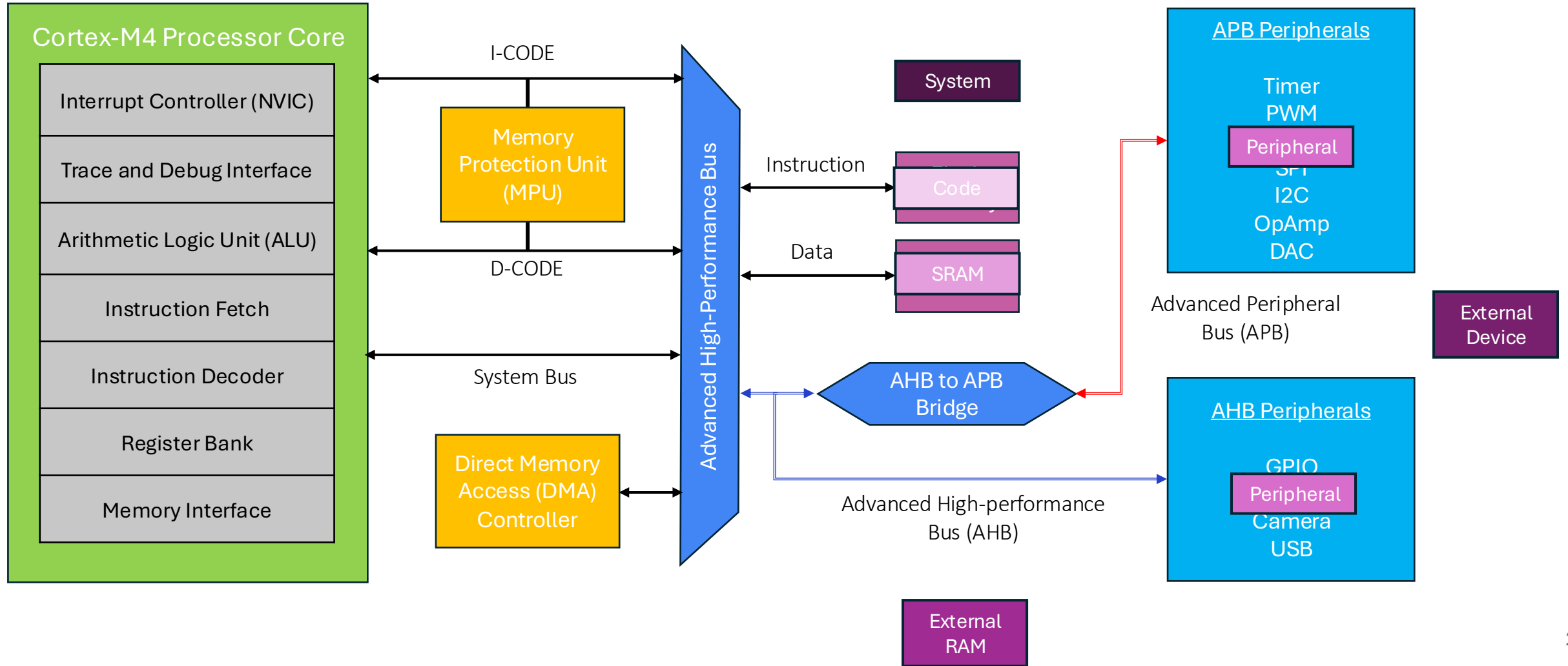
Cortex-M4 Memory Map



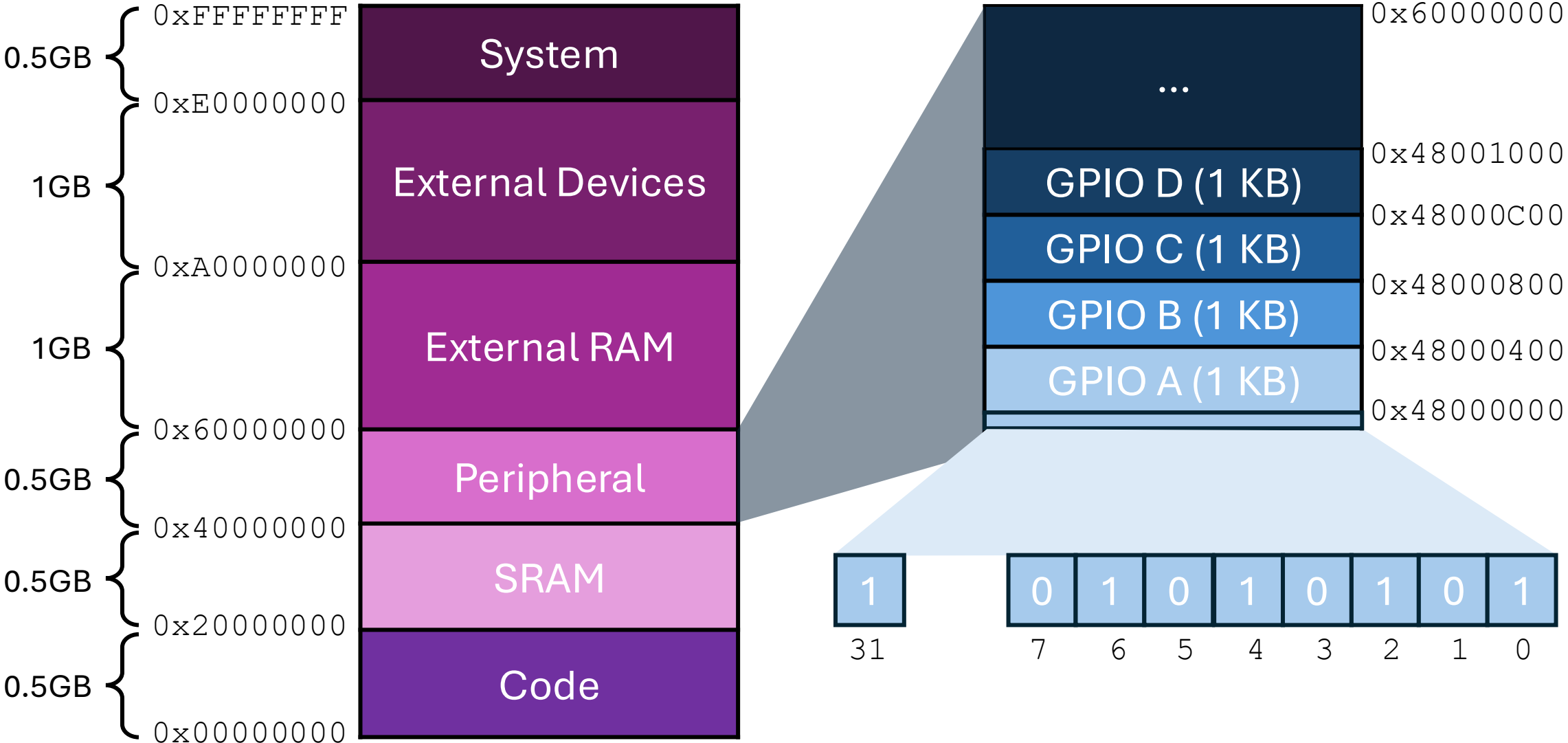
Memory blocks are distributed



Memory blocks are distributed



Memory Map for GPIO Peripheral



Basic Idea – Memory Mapped I/O

Port-mapped I/O uses special machine instructions that are designed specifically for I/O instructions

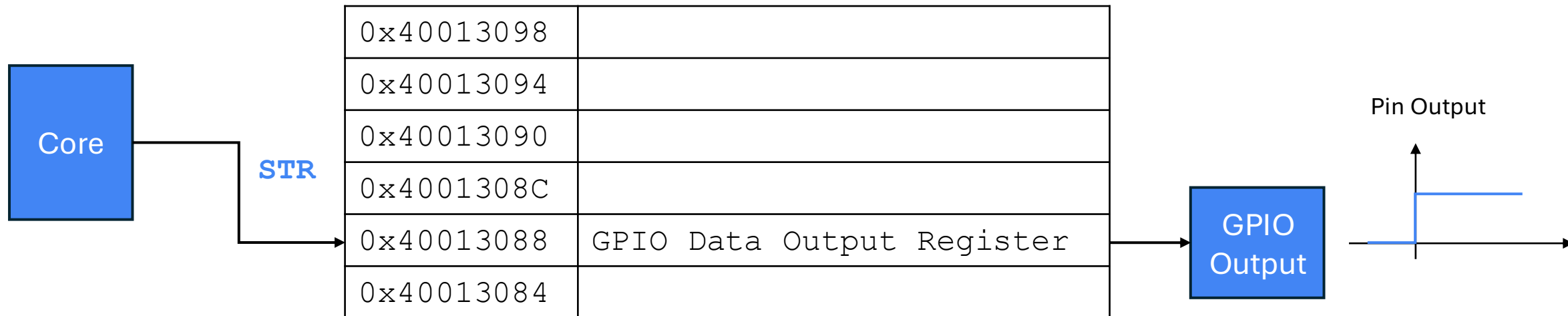
Memory-mapped I/O does not need any special instructions.

The memory and the I/O devices share the same address space.

Each peripheral register or data buffer is assigned to a memory address in the memory space of the microprocessor.

Basic Idea - Memory Mapped I/O

- A convenient way to interface with I/O devices
- Each device register is assigned to a memory address in the address space of the microprocessor
- Use native CPU load/store instructions (LDR/STR)



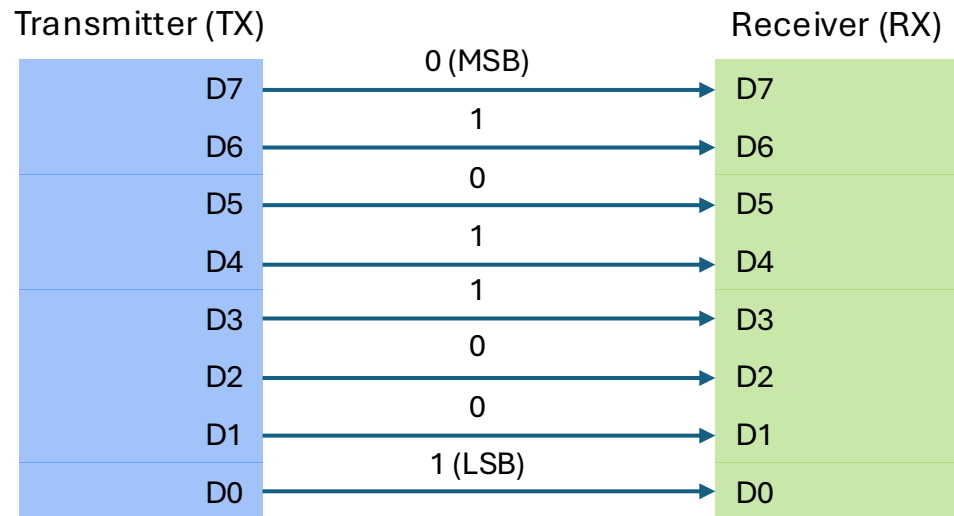
What are we learning about today?

- Deeper Dive: Stack and Application Binary Interface (ABI)
- Memory-mapped I/O (MMIO)
- **Bus Architectures**
- Advanced Microcontroller Bus Architecture (AMBA)

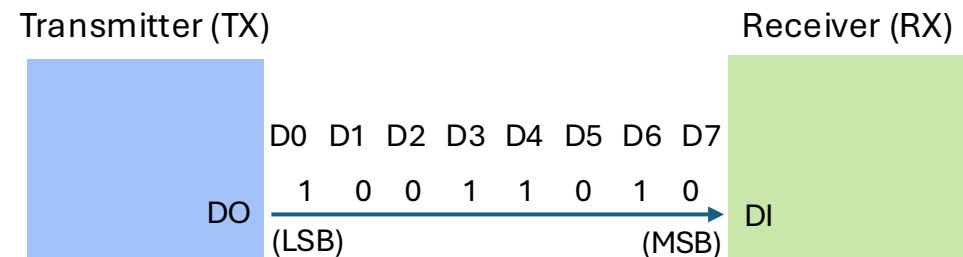
Data Bus Types

A ***data bus*** is a set of wires, supporting circuits and protocol, that provides transportation for data between two locations

Parallel Interface Example

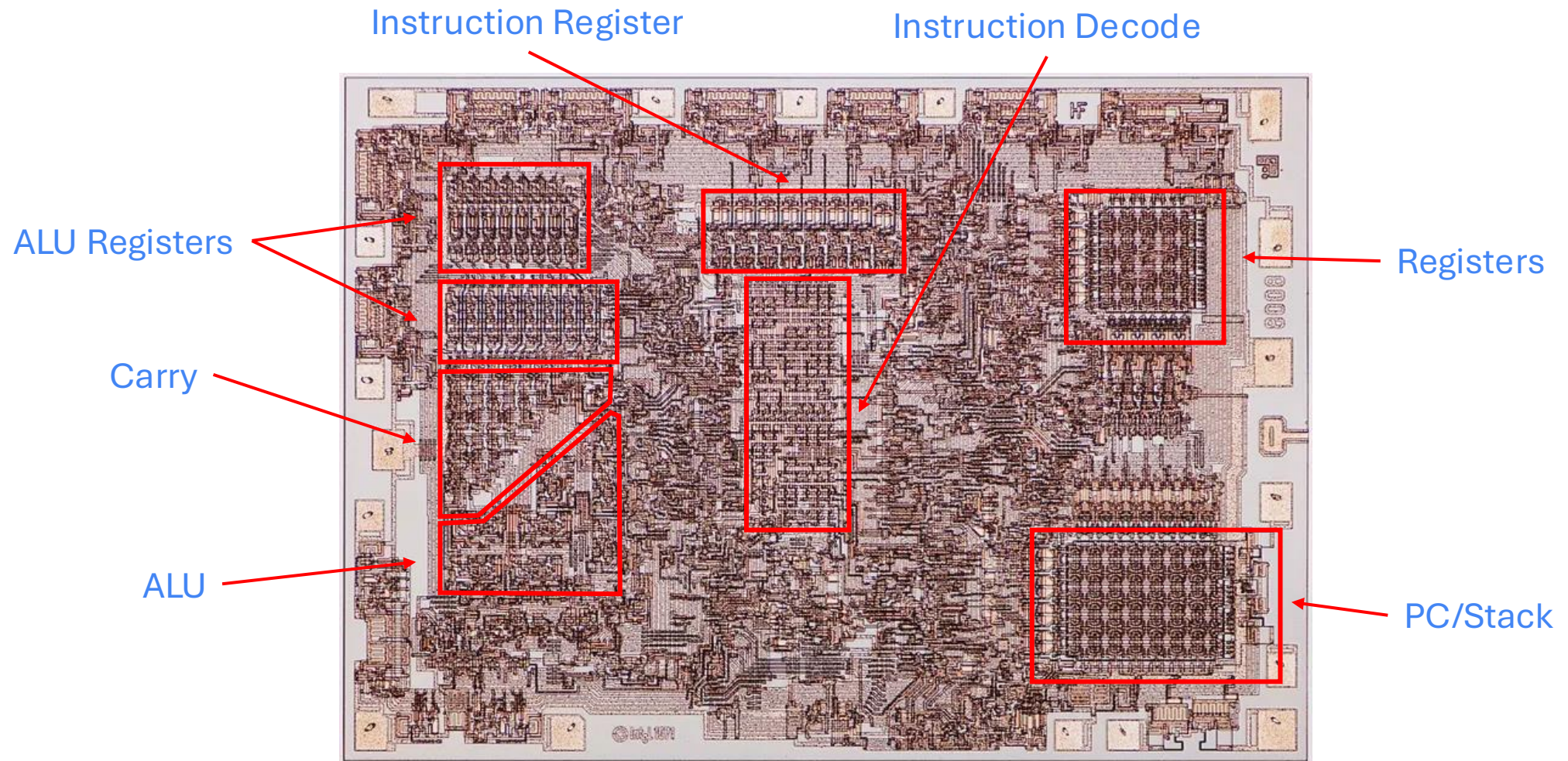


Serial Interface Example



Which type of bus is most used on chip?

Intel 8008 Microprocessor

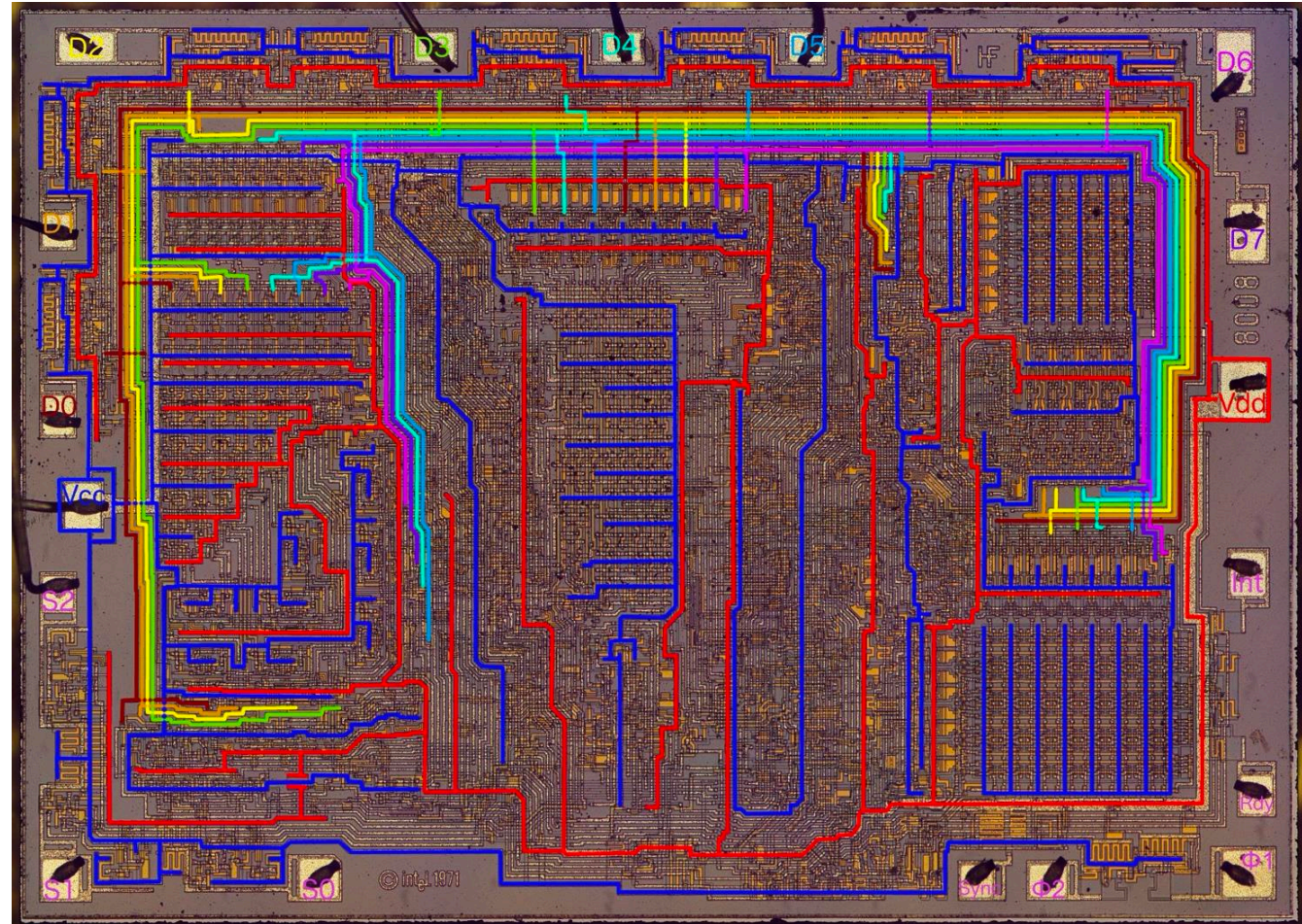


Which type of bus is most used on chip?

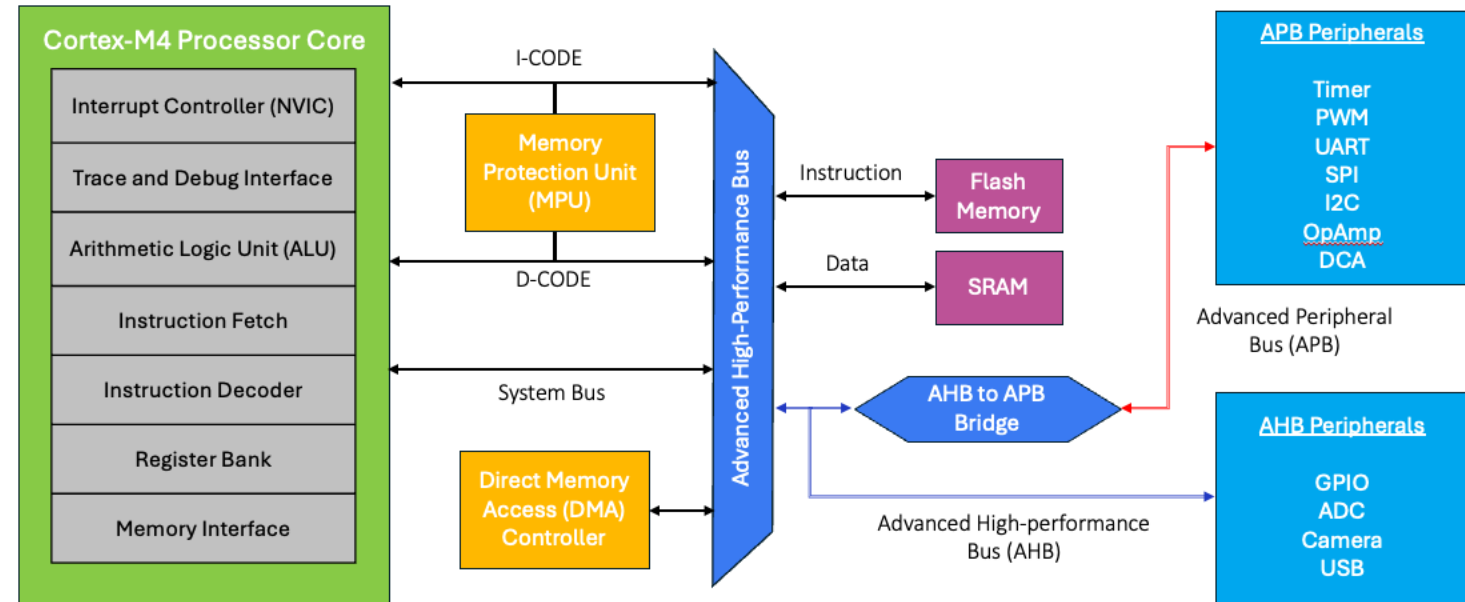
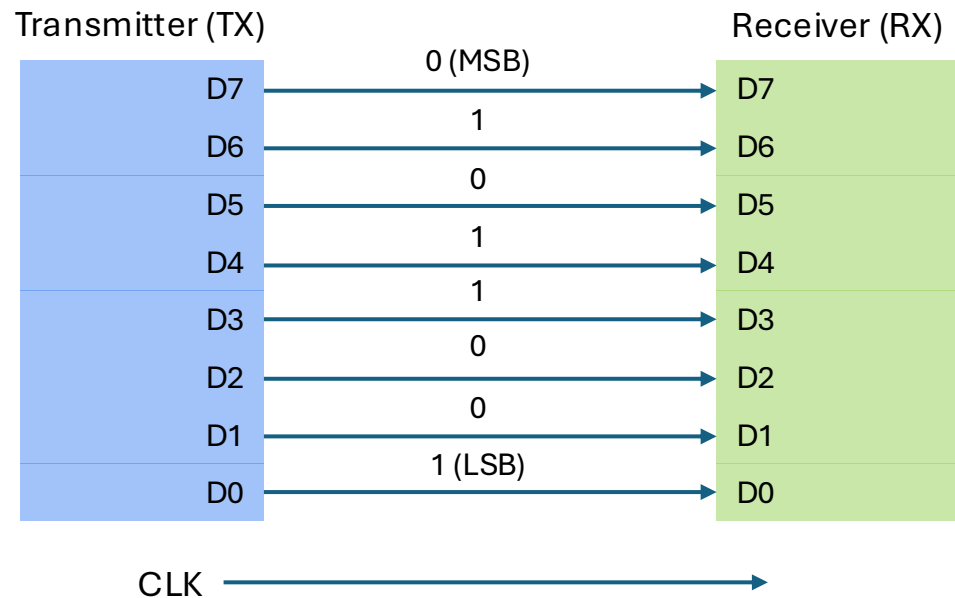
Intel 8008 Microprocessor

Power Bus: Red and Blue

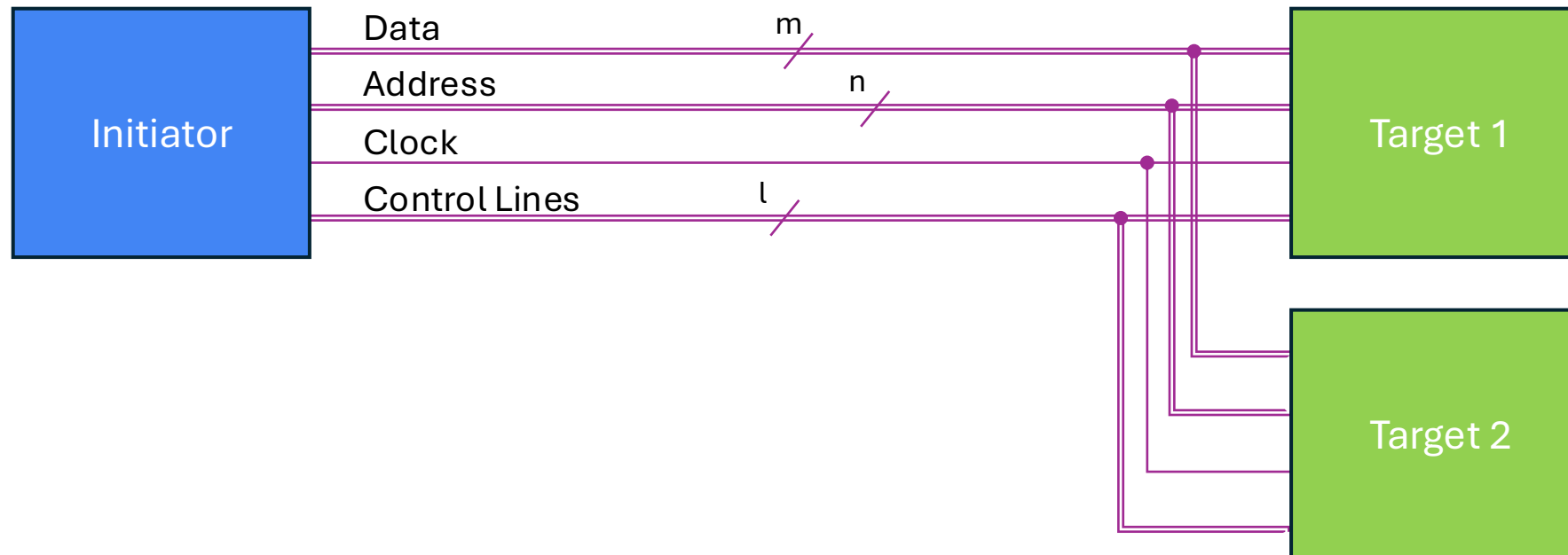
8-bit Parallel Data Bus:
Shown in 8 rainbow colors



How do we send to individual peripherals?



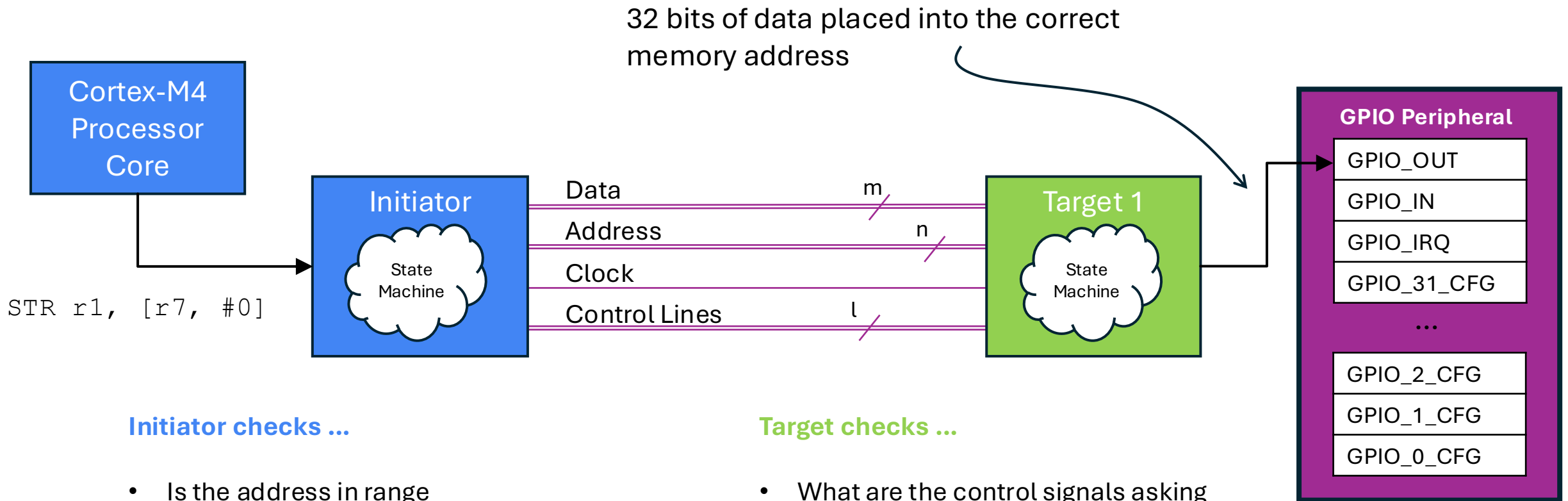
How would we implement a multi-target parallel bus? What signal lines might you want to add beyond data and clock?



Bus Terminology

- A transaction is between an *initiator* and a *target*
- An **Initiator** can start a transaction event and controls which devices receive and transmit data.
- A **Target** device can only transmit when commanded to by the Initiator.

How do we use buses on the Cortex-M4?



Initiator checks ...

- Is the address in range
- Load or Store
- Control and timing signals are needed

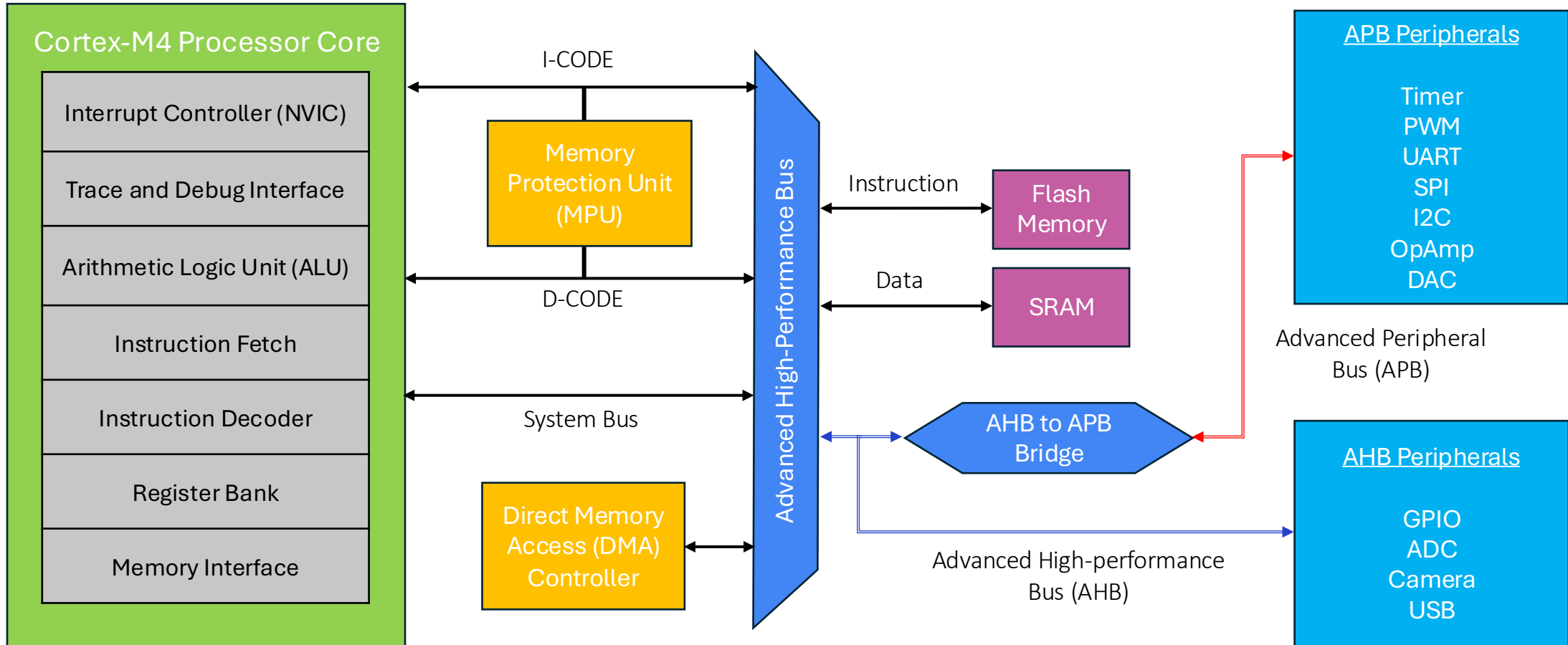
Target checks ...

- What are the control signals asking for
- Receiving data → what to do with it?
- Transmitting data --> where do I get it from?

What are we learning about today?

- Deeper Dive: Stack and Application Binary Interface (ABI)
- Memory-mapped I/O (MMIO)
- Bus Architectures
- **Advanced Microcontroller Bus Architecture (AMBA)**

Advanced Microcontroller Bus Architecture (AMBA)



Advanced Microcontroller Bus Architecture (AMBA)

EE 186

[Home](#)
[Course Overview](#)
[Course Schedule](#)
[Assignments](#)
[Resources](#)
[Staff](#)

ARM

[ARMv7 Architecture Reference Manual](#)

[AMBA Advanced High-performance Bus \(AHB\) Protocol Specification](#)

[AMBA Advanced Peripheral Bus \(APB\) Protocol Specification](#)

STM32

[STM32 Reference Manual](#)

[STM32 Datasheet](#)

[STM32 IDE](#)

[STM32 Videos](#)

[Nucleo-144 User Manual](#)

[Nucleo Board Pinout](#)

[Nucleo Development Board Website](#)

**AMBA[®] APB
Protocol Specification**

arm

Copyright © 2003-2021 Arm Ltd. All rights reserved.
ARM IHI 0024D (ID041221)

Advanced Microcontroller Bus Architecture (AMBA)

An on-chip communication standard for embedded microcontrollers

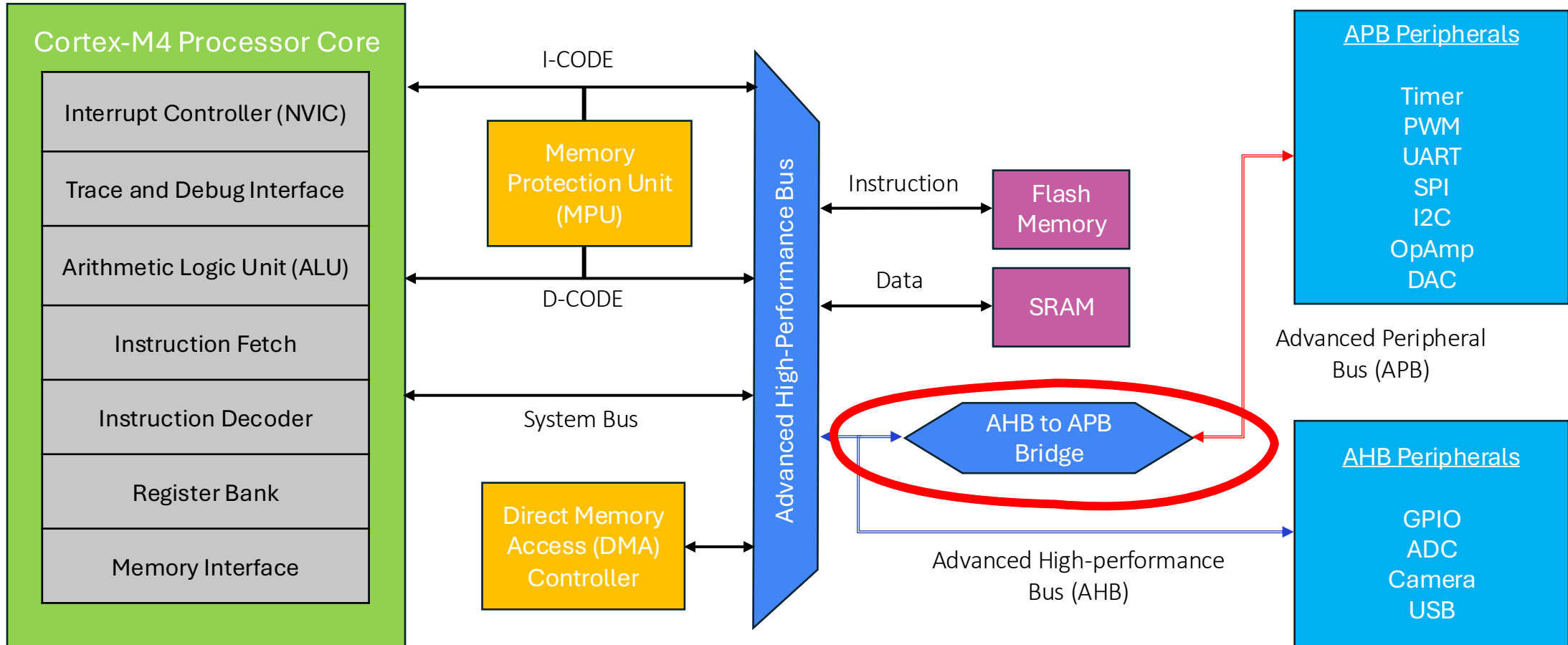
Advanced High-performance Bus (AHB)

- High performance
- Pipelined operation
- Burst transfer
- Multiple bus initiators
- Split transactions

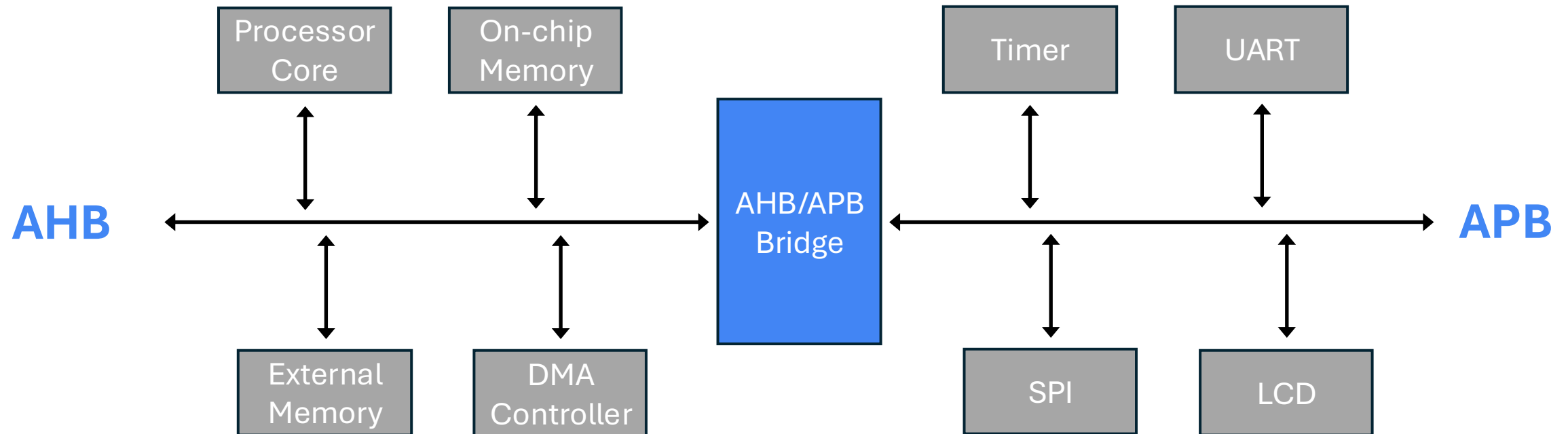
Advanced Peripheral Bus (APB)

- Low power
- Simple interface
- Latched address/control
- Suitable for many peripherals

Advanced Microcontroller Bus Architecture (AMBA)

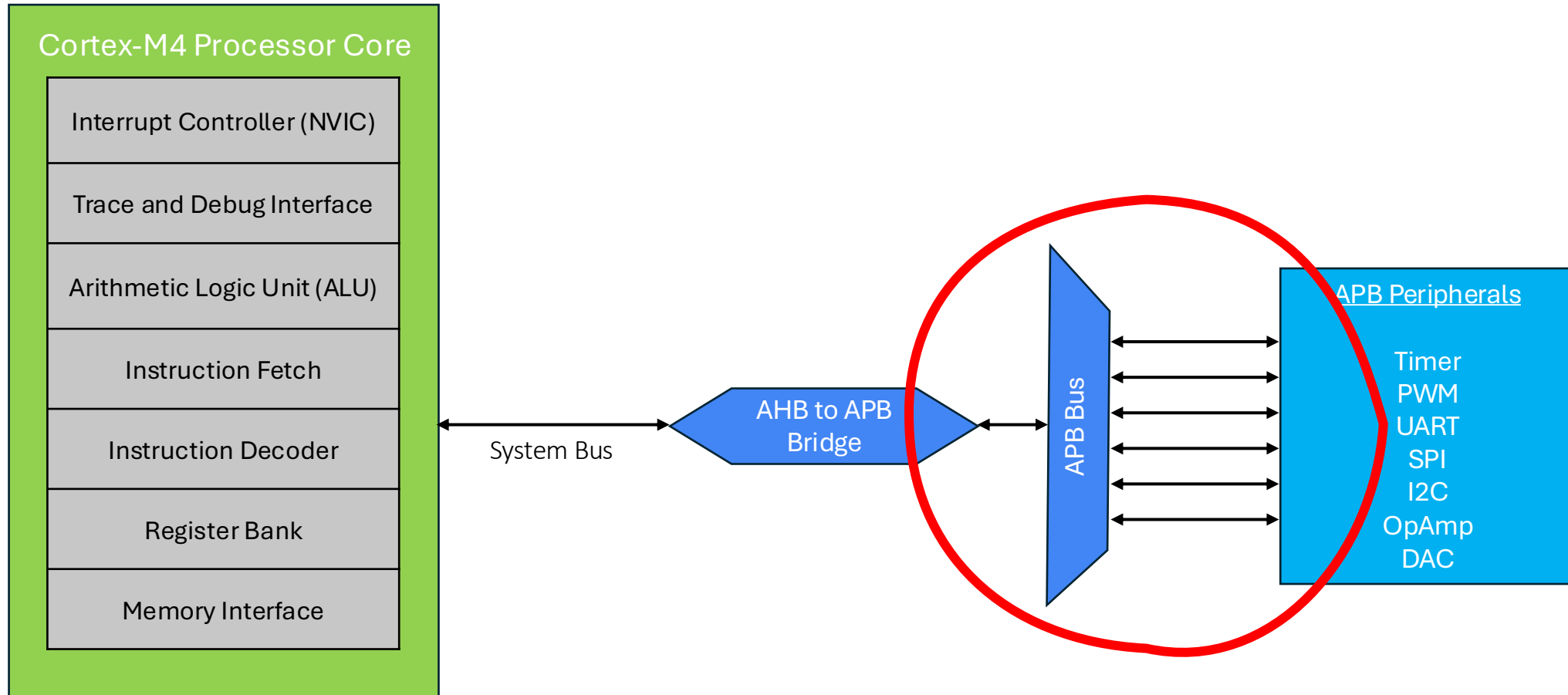


Advanced Microcontroller Bus Architecture (AMBA)



- A bus matrix ensures no modules block data transmissions for long durations and arbitrates conflicts (e.g., multiple access attempts)
- A bridge is required to connect the AHB and APB buses
 - Allows an *initiator* on the AHB bus to communicate with a peripheral device on the APB bus

How do we interface with individual peripherals?



Some APB notation you will see

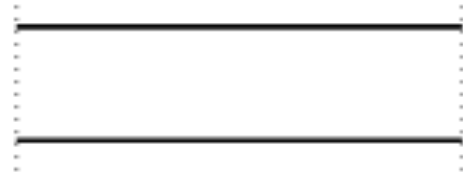
Clock



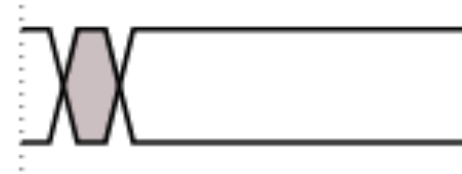
HIGH to LOW



Bus Stable

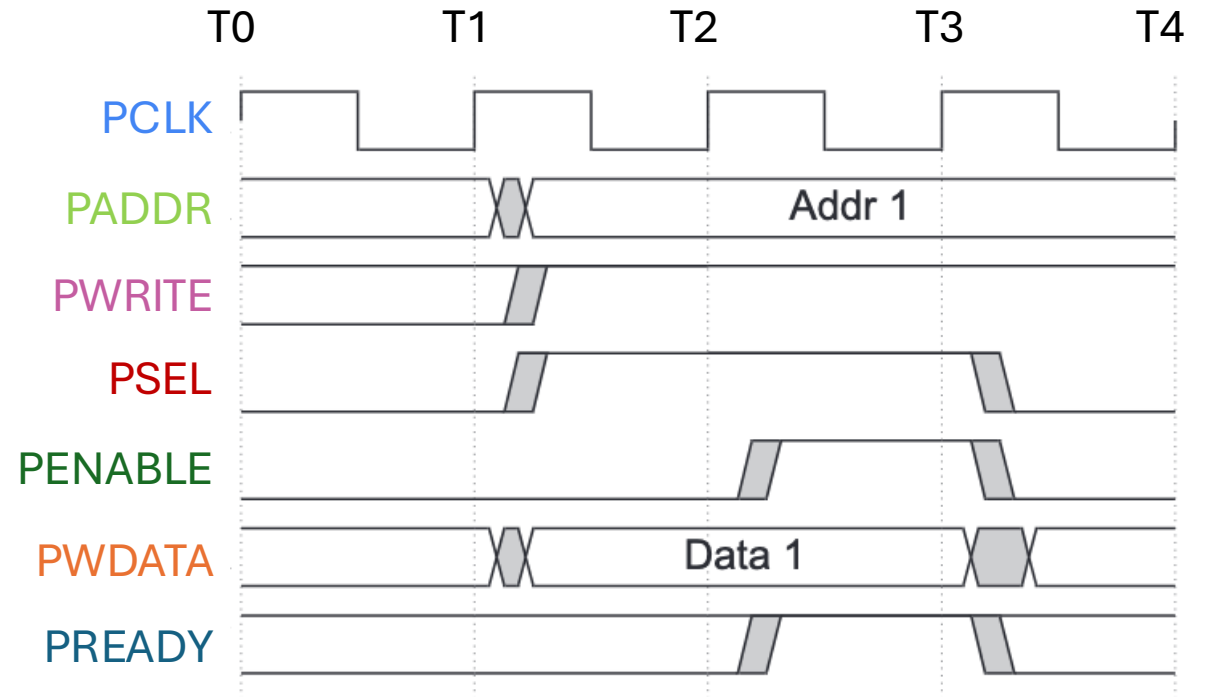


Bus Change



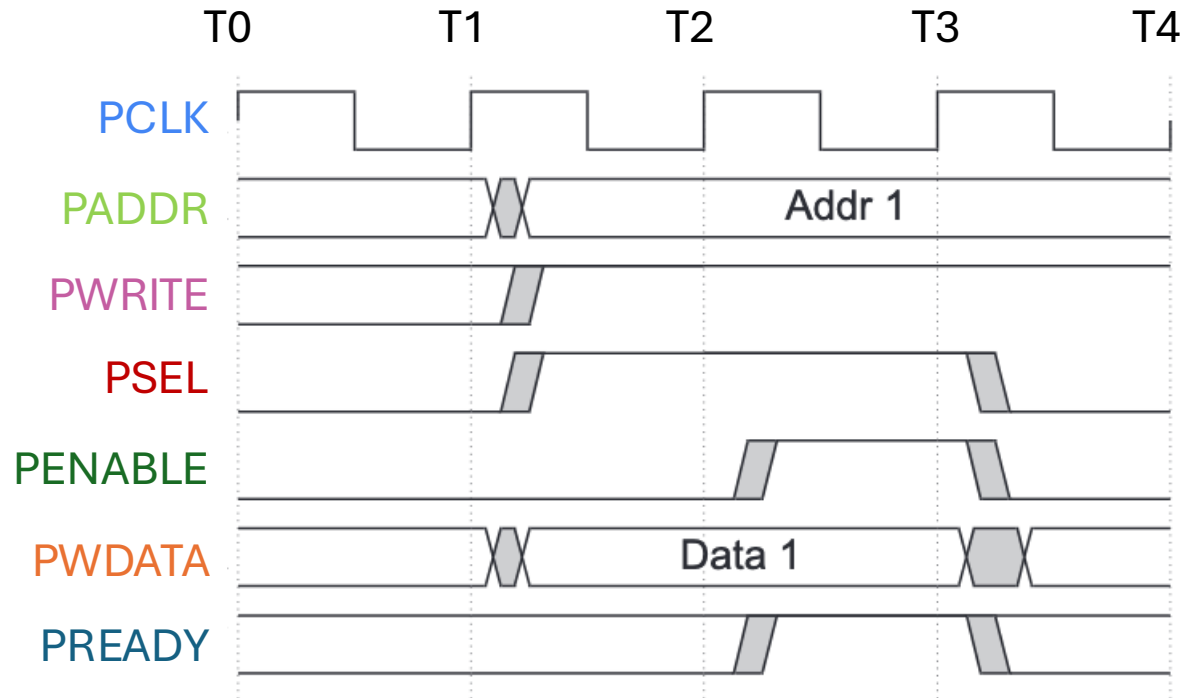
APB Bus Signals

- **PCLK – [1 bit]**
 - Clock ~pos edge
- **PADDR – [32 bits]**
 - Address on the bus
 - Up to 32 bits but not always
- **PWRITE – [1 bit]**
 - 1 = Write
 - 0 = Read
- **PWDATA – [32 bits]**
 - Data written to the I/O device

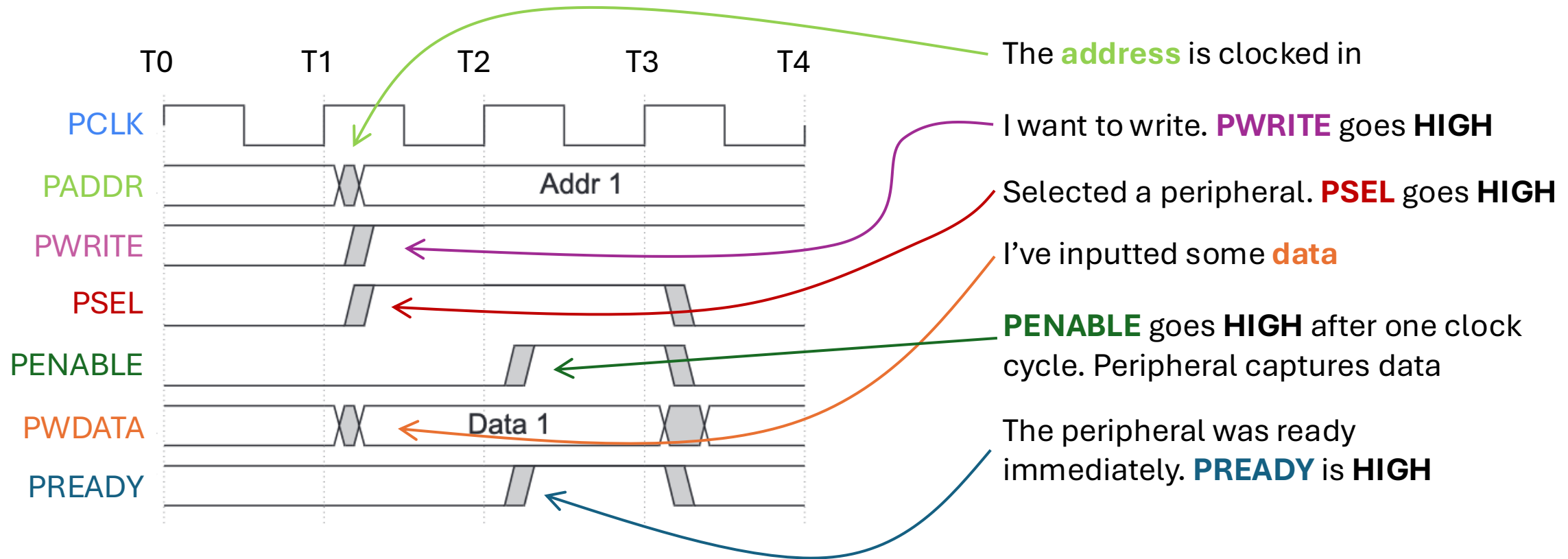


APB Bus Signals

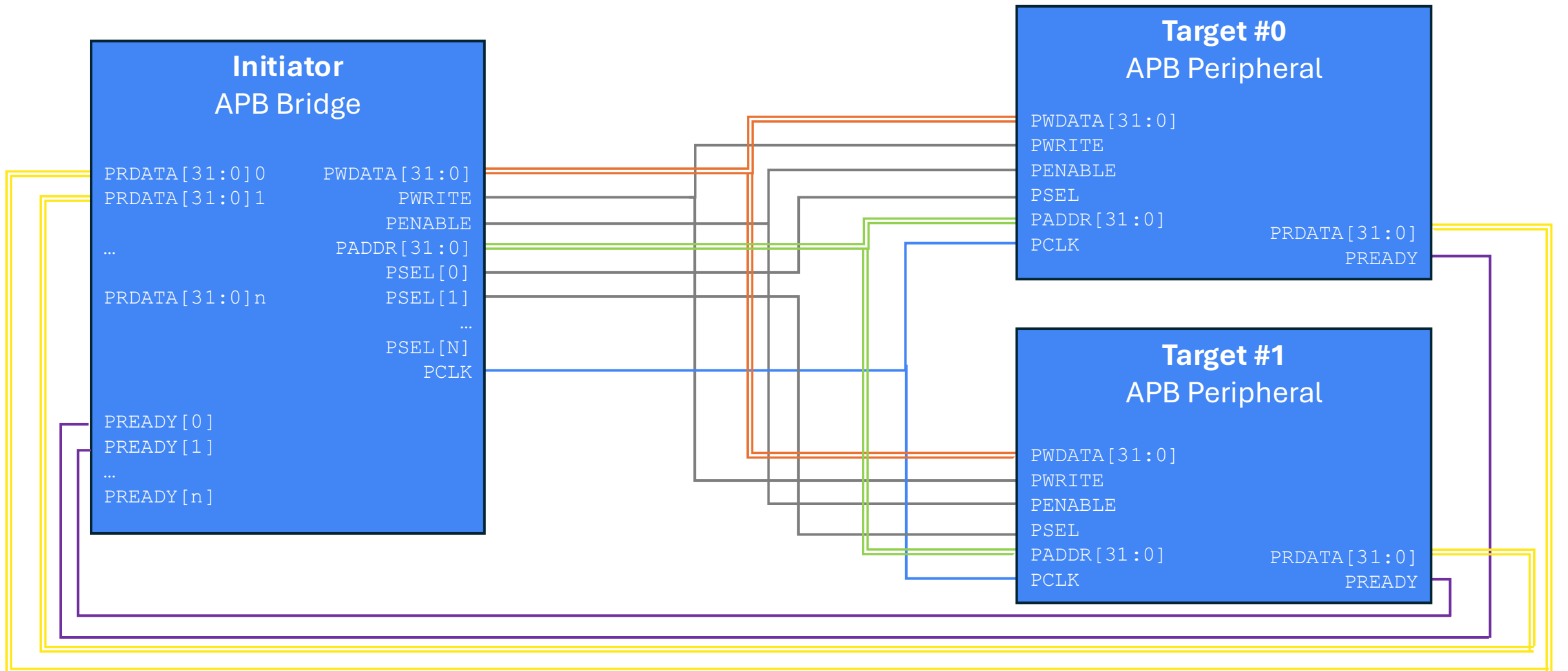
- **PSEL**
 - Asserted if the current bus transaction is targeted to **this device**
 - Each target has a PSEL
- **PENABLE**
 - High during entire transaction other than the first cycle
- **PREADY**
 - Driven by the target
 - Indicates if the target is **ready** to do the transaction
 - Each target has a PREADY
- **PRDATA [32 bits]**
 - (not on timing diagram)
 - Read data from bus signals



What's happening here?



APB: One Initiator, Multiple Targets



Example: Driving Multiple Targets

Let's assume we have one APB Initiator and two Target devices (D1 and D2)

- D1 is mapped to addresses `0x00001000` – `0x0000100F`
- D2 is mapped to addresses `0x00001010` – `0x0000101F`

The Cortex-M4 processor Core executes the following instructions:

`STR R0, [R1, #4]`

`R0 = 0x00C0FFEE`

`R1 = 0x00001000`

- this is an address that points to a memory location
- this offset **does not change** the starting address held in R1
- `address = base + offset = 0x00001000 + 4 = 0x00001004`

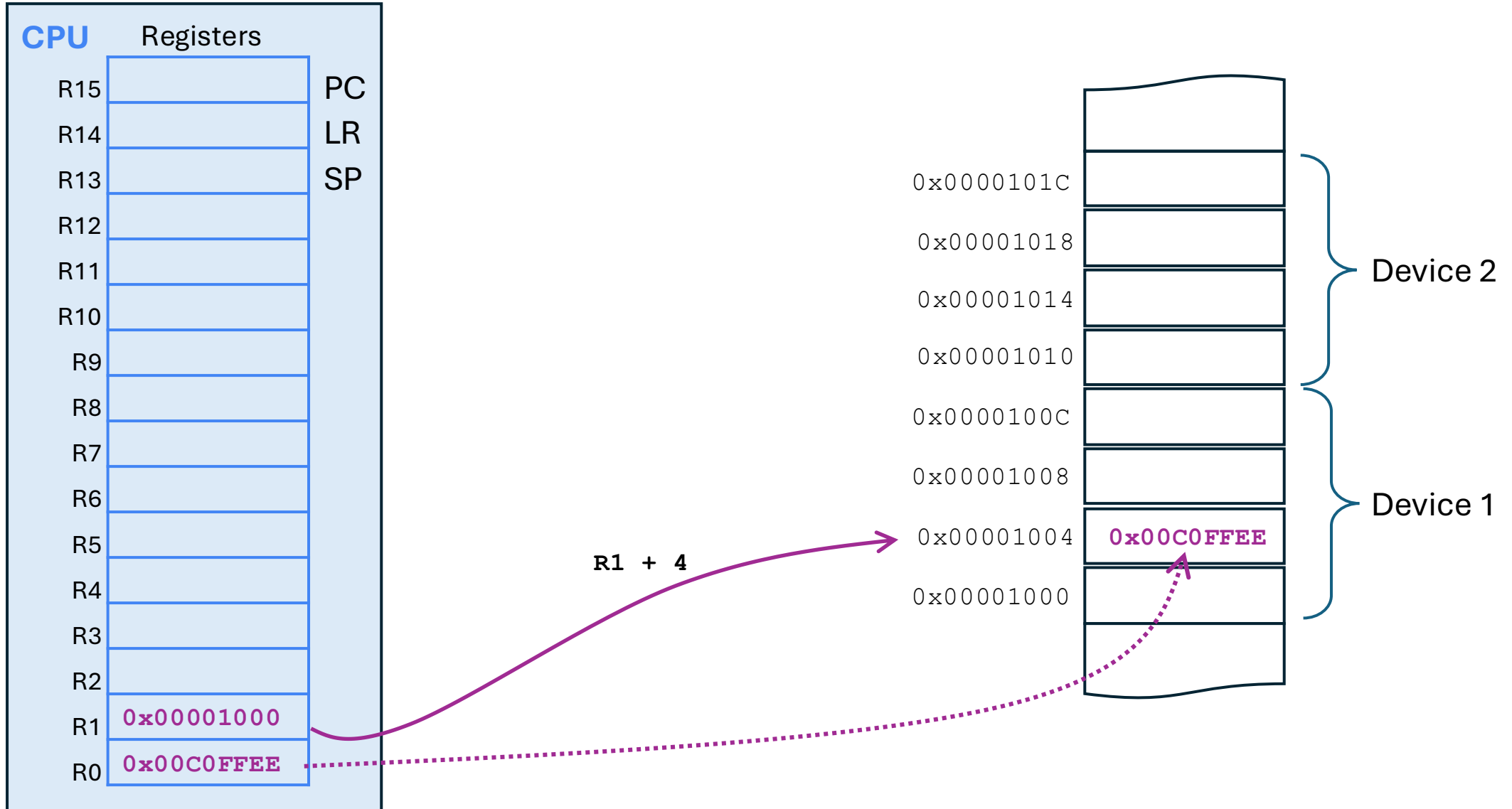
Example: Driving Multiple Targets

Execute:

```
STR R0, [R1, #4]
```

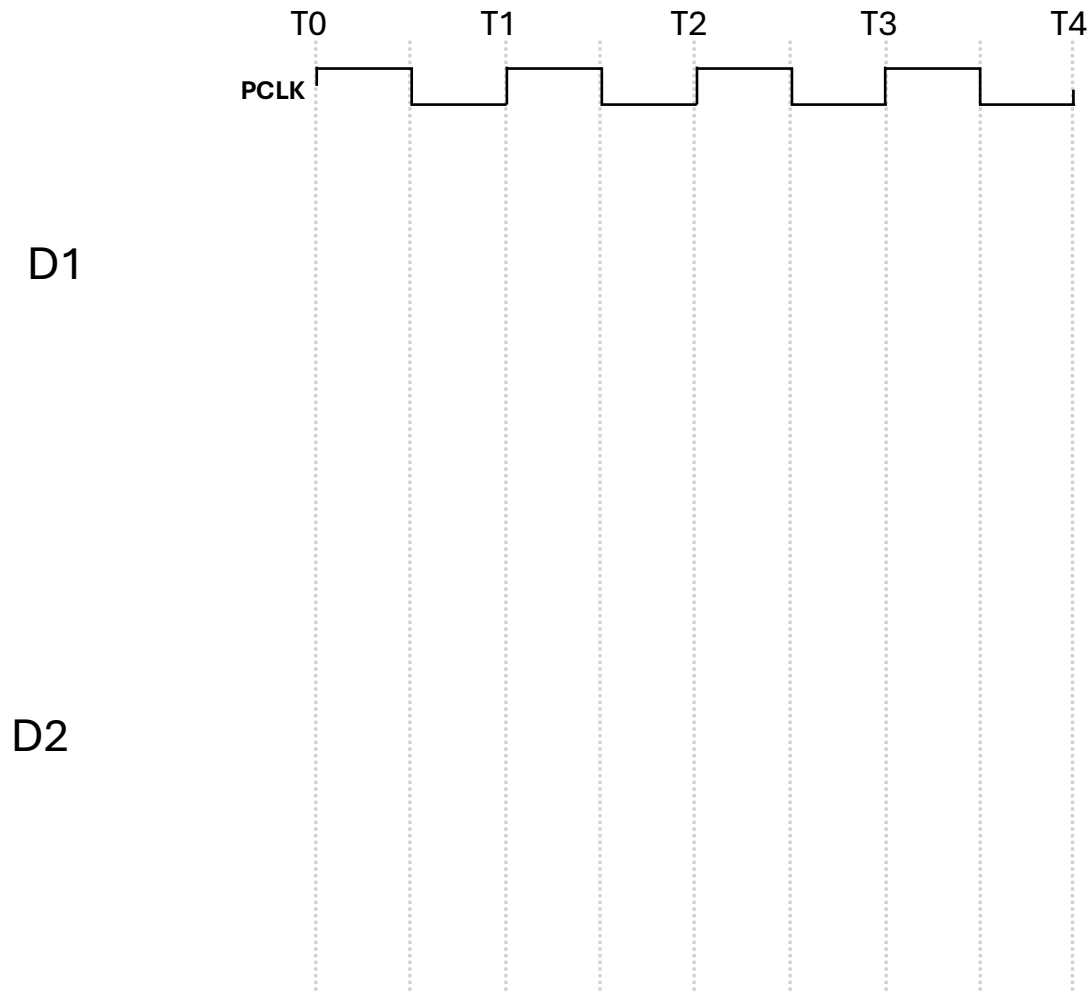
Given:

R0 = 0x00C0FFEE
R1 = 0x00001000



Group Exercise:

Show the signals for both devices



Let's assume we have one APB Initiator and two Target devices (D1 and D2)

- D1 is mapped to addresses `0x00001000 - 0x0000100F`
- D2 is mapped to addresses `0x00001010 - 0x0000101F`

The Cortex-M4 processor Core executes the following instructions:

Execute:

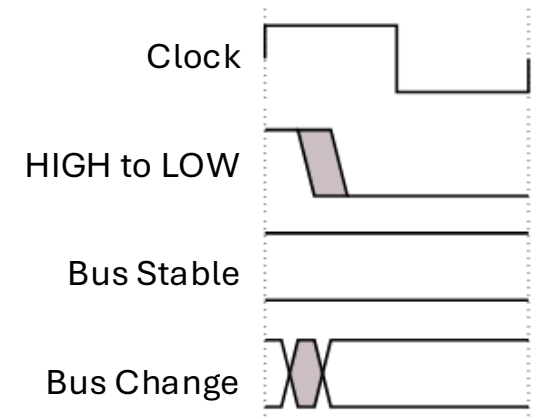
```
STR R0, [R1, #4]
```

Given:

```
R0 = 0x00C0FFEE
```

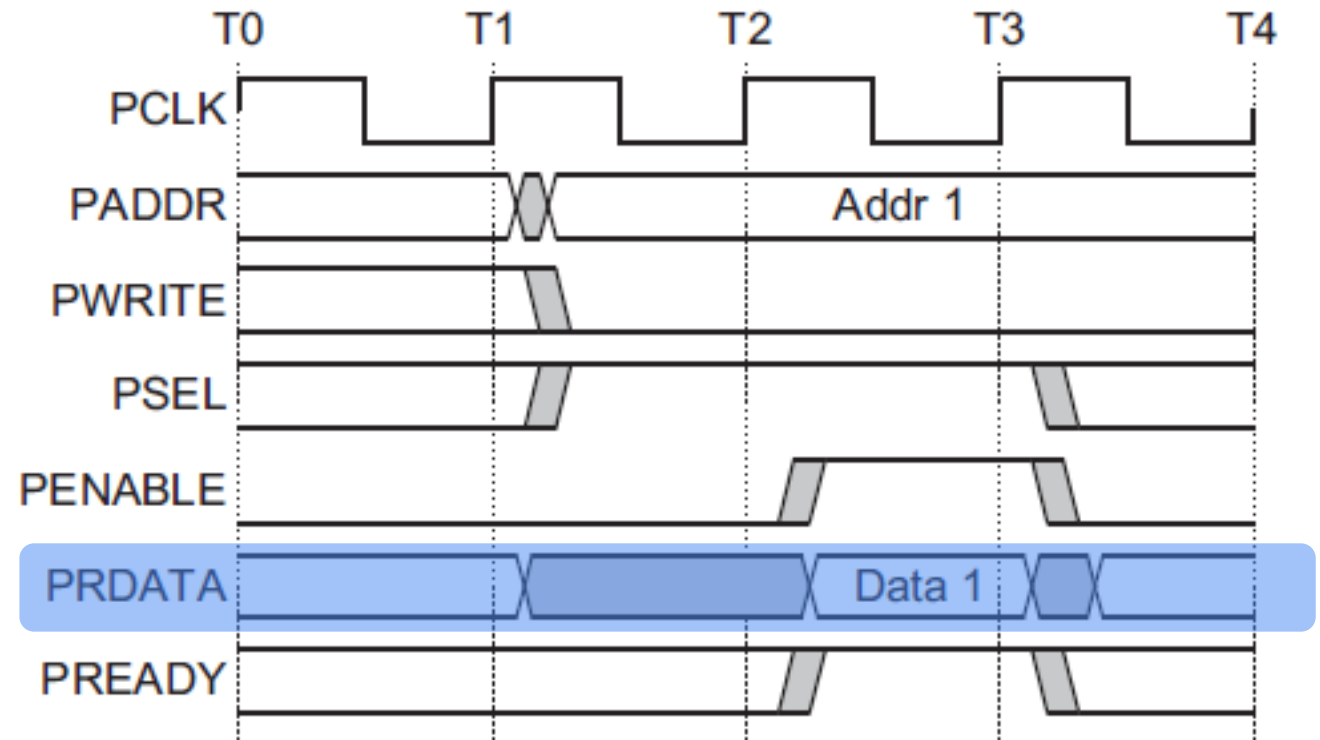
```
R1 = 0x00001000
```

Remember →

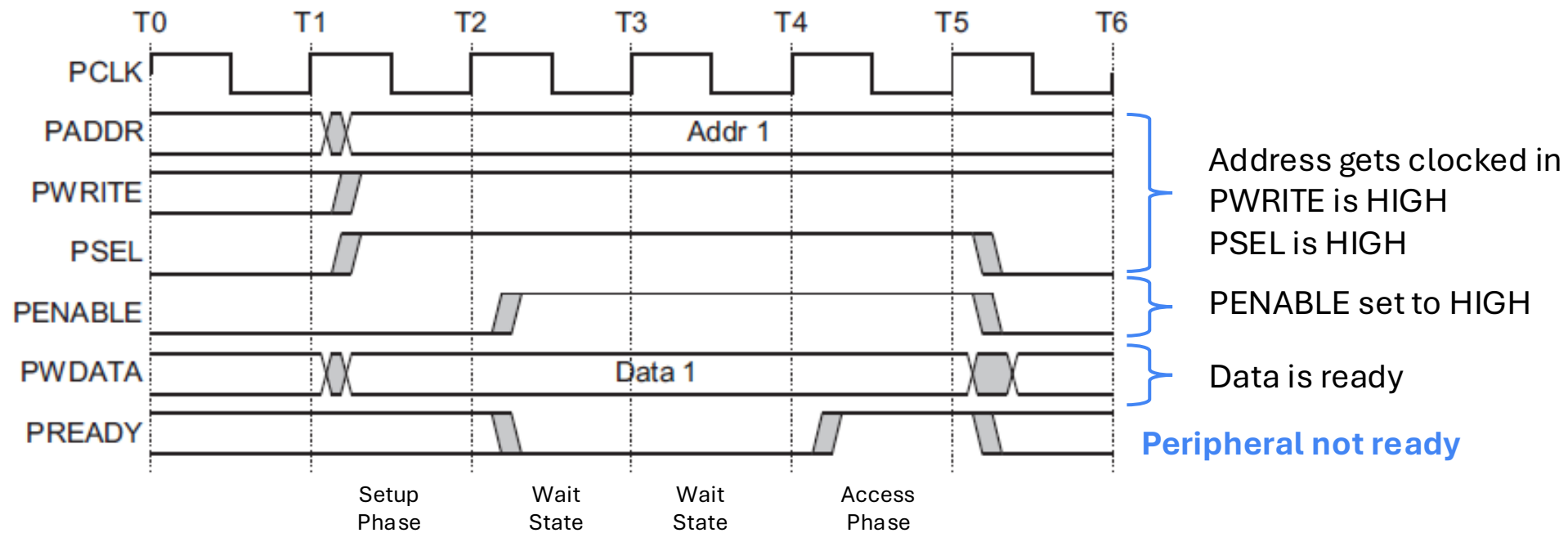


What about APB reads?

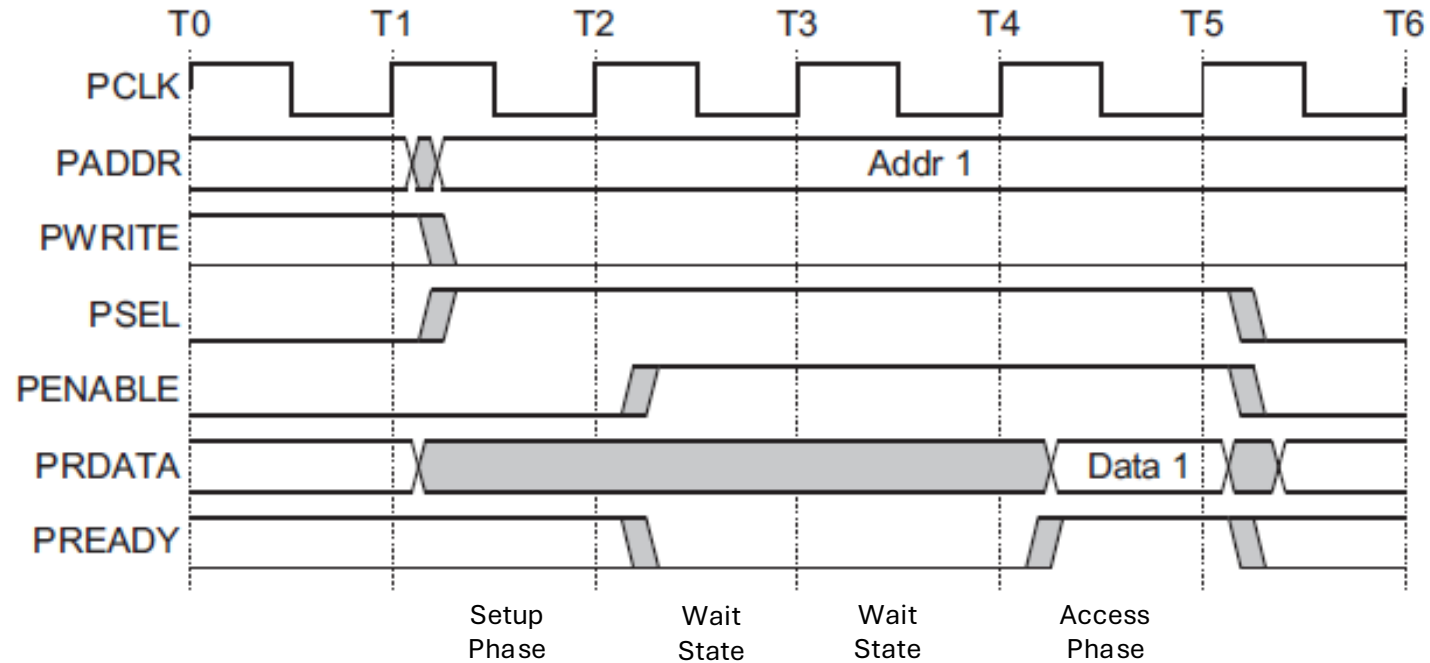
Each target device has its own read data (PRDATA) bus!



A **write** transfer with wait states



A read transfer with wait states



APB State Machine

IDLE

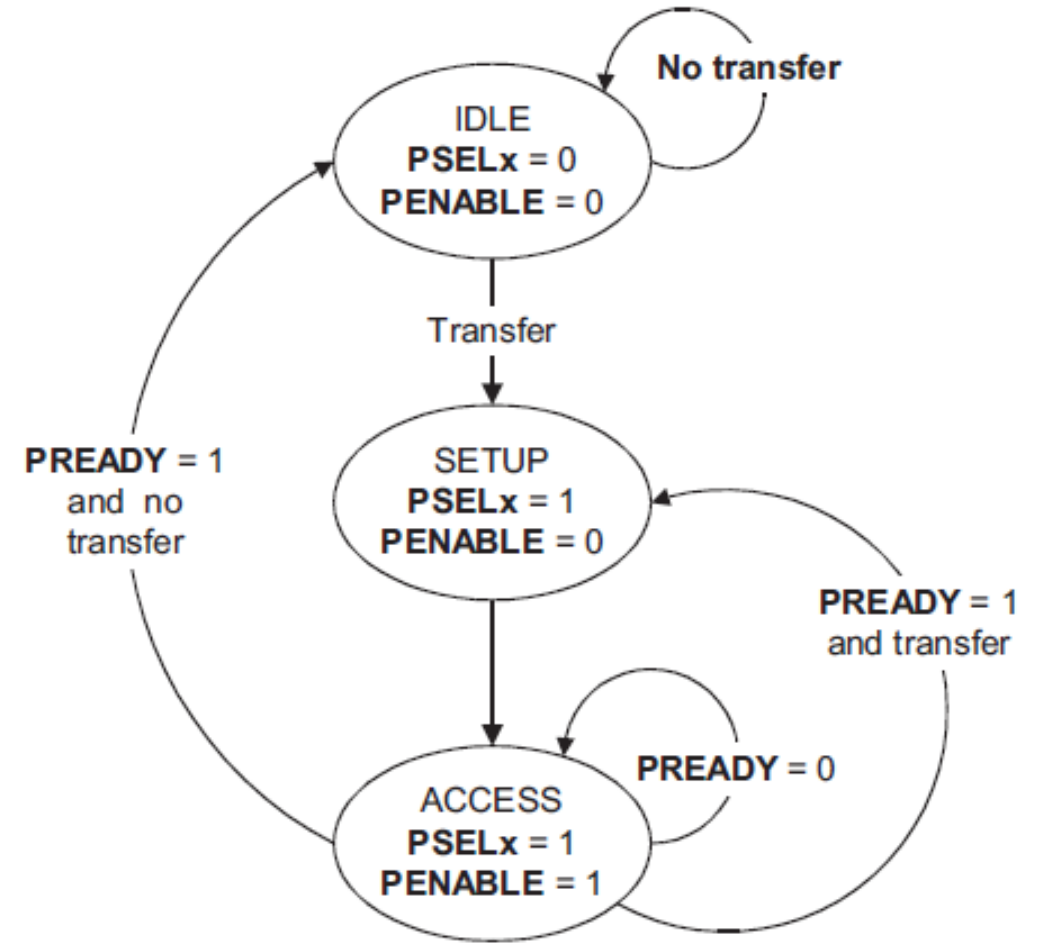
- The default APB state

SETUP

- When a transfer is required
- PSELx is asserted
- Only one clock cycle

ACCESS

- PENABLE is asserted
- Addr, write, select, and write data remain stable
- Stay if PREADY = 0 (LOW)
- Go to IDLE if PREADY = 1 (HIGH) & no more data
- Go to SETUP if PREADY = 1 (HIGH) & more data is pending



Recap: APB Signals

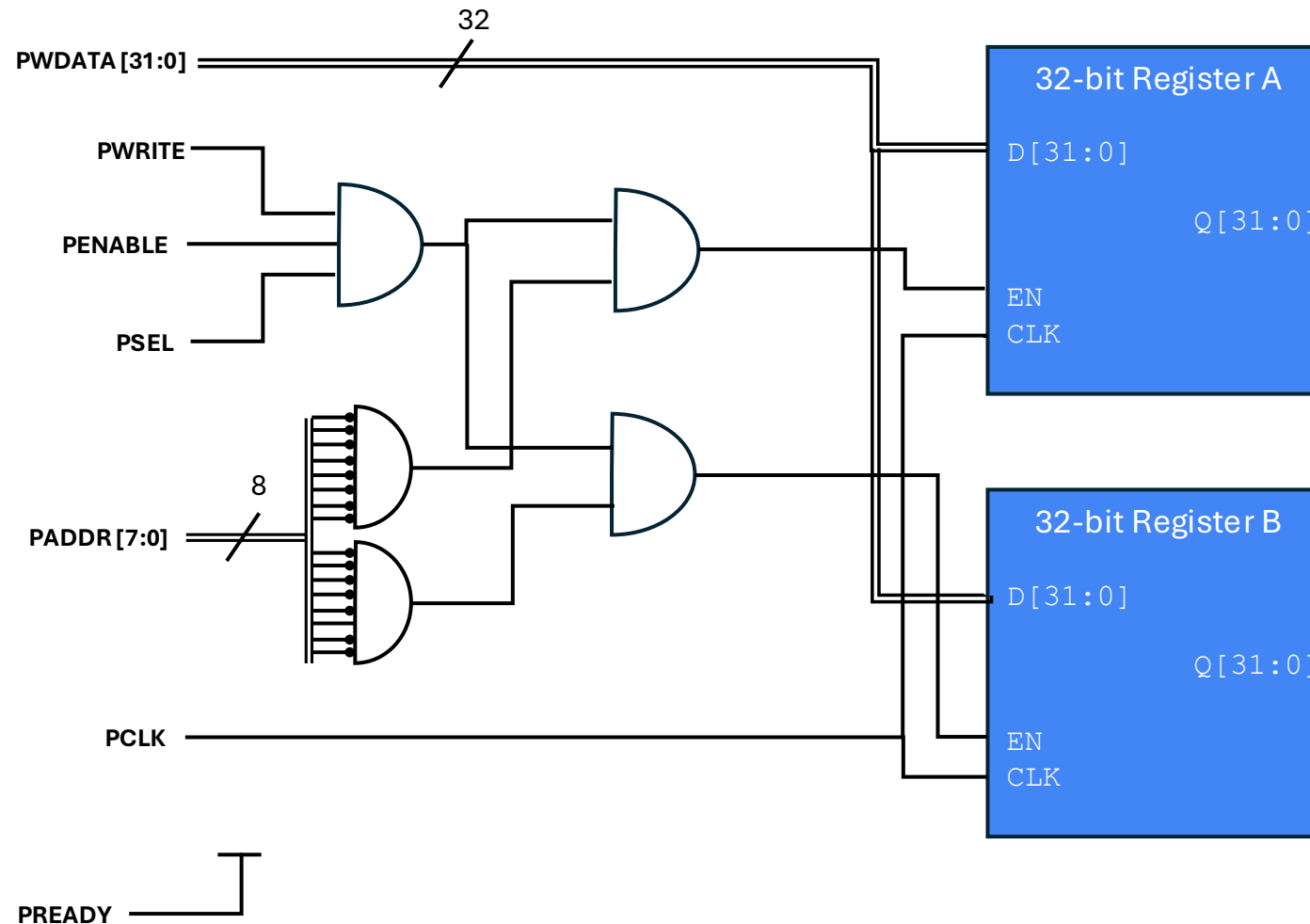
- Documentation describes all APB signals
- References on class webpage

Table 2-1 APB signal descriptions

Signal	Source	Description
PCLK	Clock source	Clock. The rising edge of PCLK times all transfers on the APB.
PRESETn	System bus equivalent	Reset. The APB reset signal is active LOW. This signal is normally connected directly to the system bus reset signal.
PADDR	APB bridge	Address. This is the APB address bus. It can be up to 32 bits wide and is driven by the peripheral bus bridge unit.
PPROT	APB bridge	Protection type. This signal indicates the normal, privileged, or secure protection level of the transaction and whether the transaction is a data access or an instruction access.
PSELx	APB bridge	Select. The APB bridge unit generates this signal to each peripheral bus slave. It indicates that the slave device is selected and that a data transfer is required. There is a PSELx signal for each slave.
PENABLE	APB bridge	Enable. This signal indicates the second and subsequent cycles of an APB transfer.
PWRITE	APB bridge	Direction. This signal indicates an APB write access when HIGH and an APB read access when LOW.
PWDATA	APB bridge	Write data. This bus is driven by the peripheral bus bridge unit during write cycles when PWRITE is HIGH. This bus can be up to 32 bits wide.
PSTRB	APB bridge	Write strobes. This signal indicates which byte lanes to update during a write transfer. There is one write strobe for each eight bits of the write data bus. Therefore, PSTRB[n] corresponds to PWDATA[(8n + 7):(8n)] . Write strobes must not be active during a read transfer.
PREADY	Slave interface	Ready. The slave uses this signal to extend an APB transfer.
PRDATA	Slave interface	Read Data. The selected slave drives this bus during read cycles when PWRITE is LOW. This bus can be up to 32-bits wide.
PSLVERR	Slave interface	This signal indicates a transfer failure. APB peripherals are not required to support the PSLVERR pin. This is true for both existing and new APB peripheral designs. Where a peripheral does not include this pin then the appropriate input to the APB bridge is tied LOW.

Example: Multiple registers in one peripheral

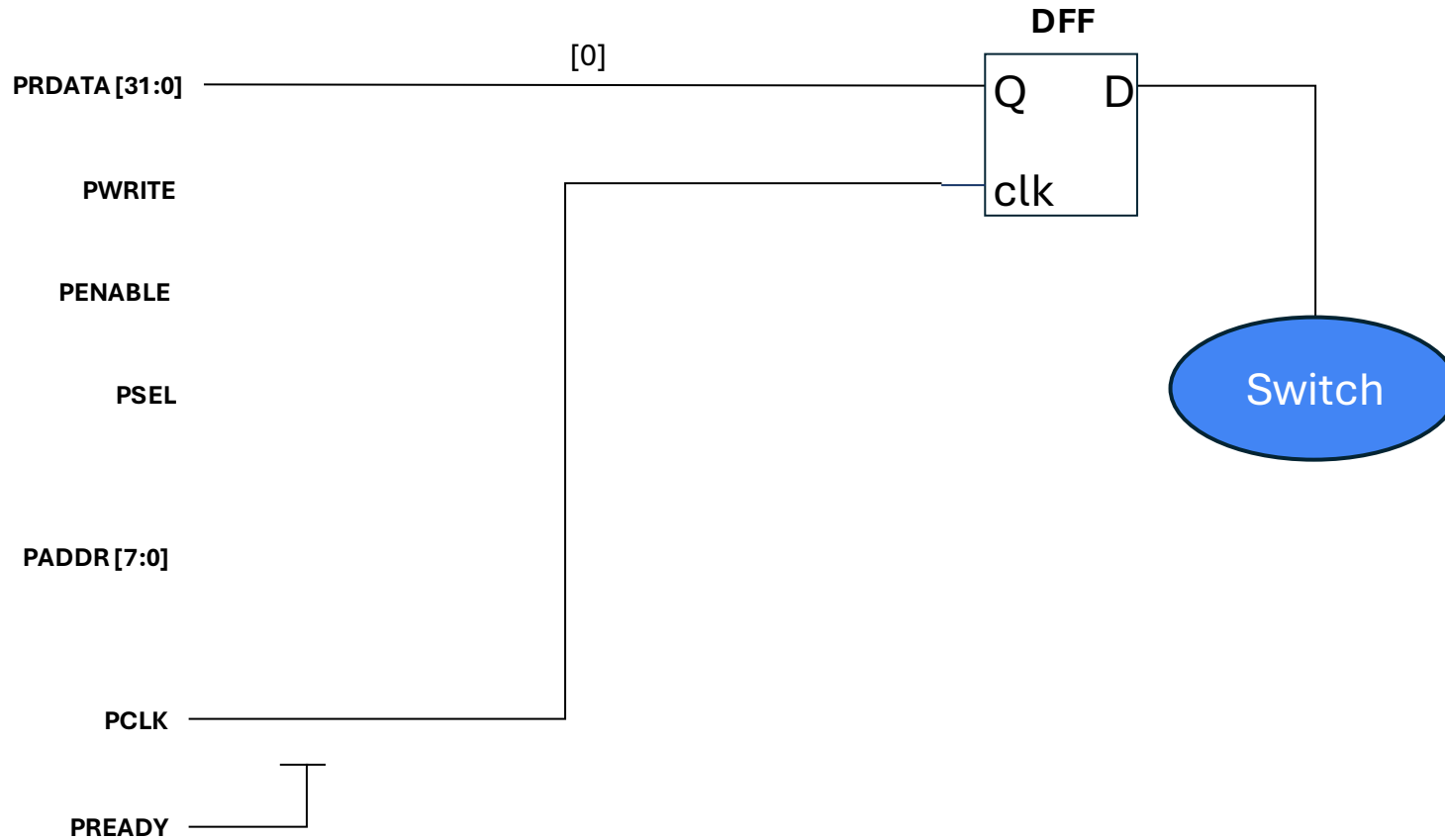
We are assuming the APB only gets the lowest 8-bits of the address



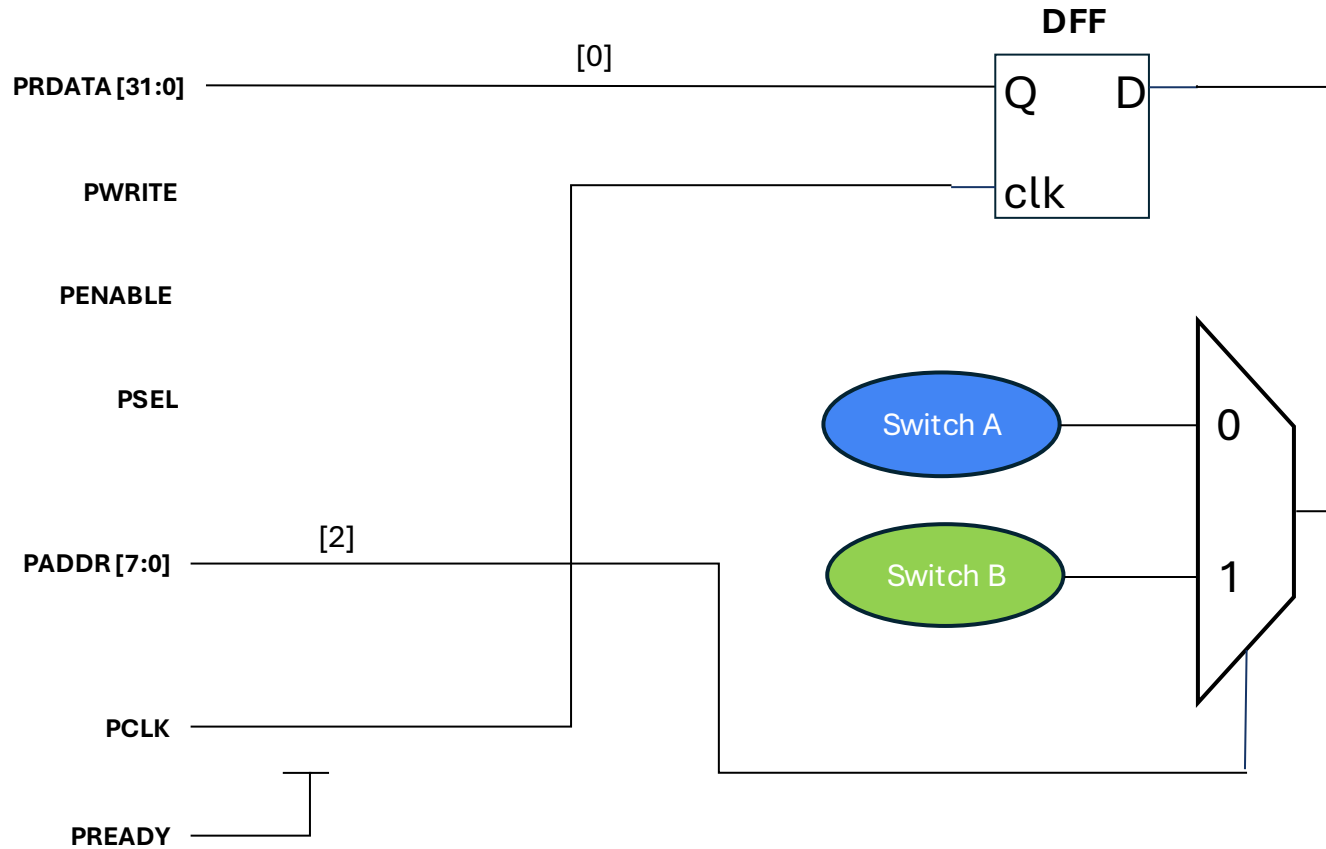
Register A should be written at address **0x00001000**

Register B should be written at address **0x00001004**

Example: Reading data from a switch



Example: Reading from a select switch



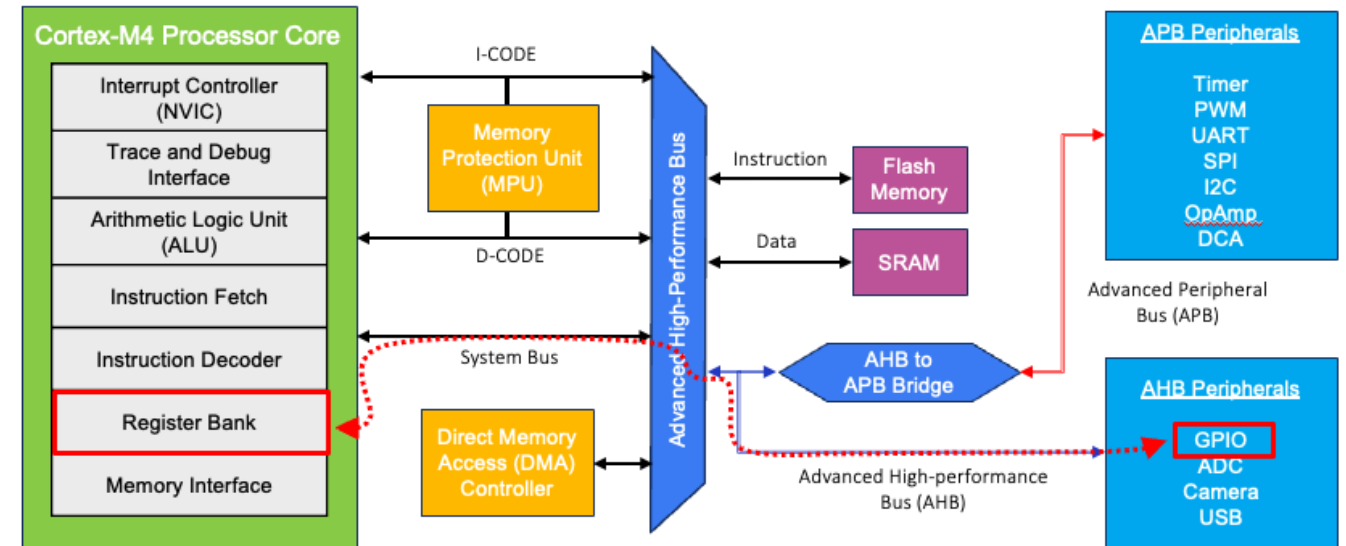
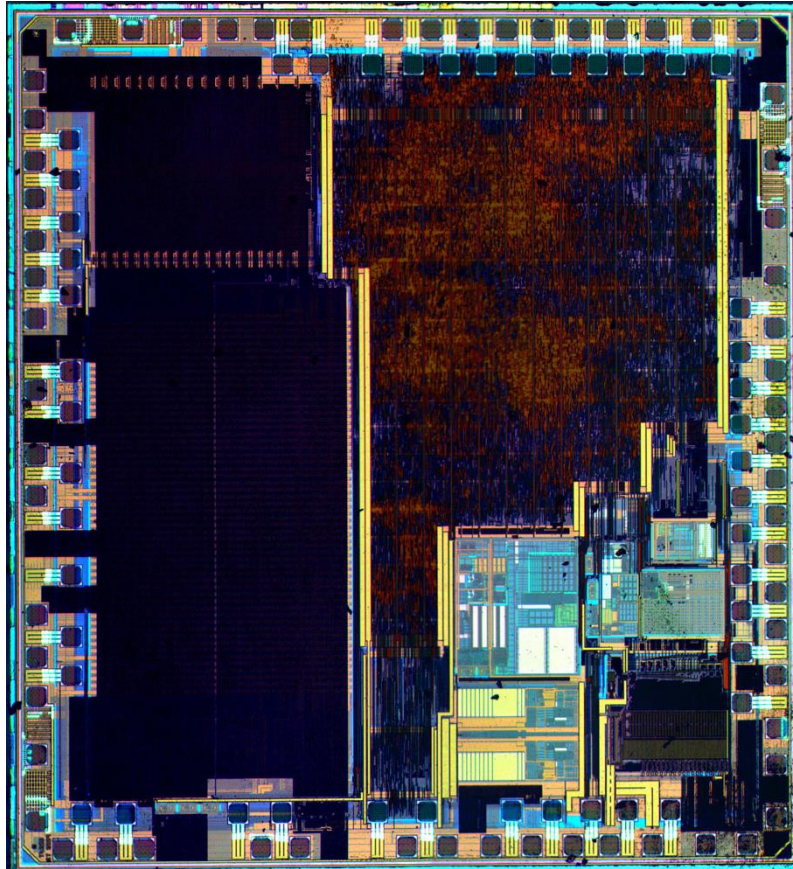
If **0x00001000** read from **A**
If **0x00001004** read from **B**

Recap: MMIO and APB

- Memory Mapped I/O
 - A way to interface with peripherals from the processor core
 - Peripherals and external devices are presented as memory and can be hardware registers or “virtual registers” via state machines
 - Use **load** and **store** instructions to access MMIO
 - The AHB and AHP transport data to-and-from the core and peripherals
- Advanced Peripheral Bus (APB)
 - Use APB to access MMIO
 - Bus structure uses address and data lines as well as control lines
 - Initiator sets PENABLE high when data on the bus is ready
 - Each Target device has its own read data bus (PRDATA) and sets PREADY high when data is ready

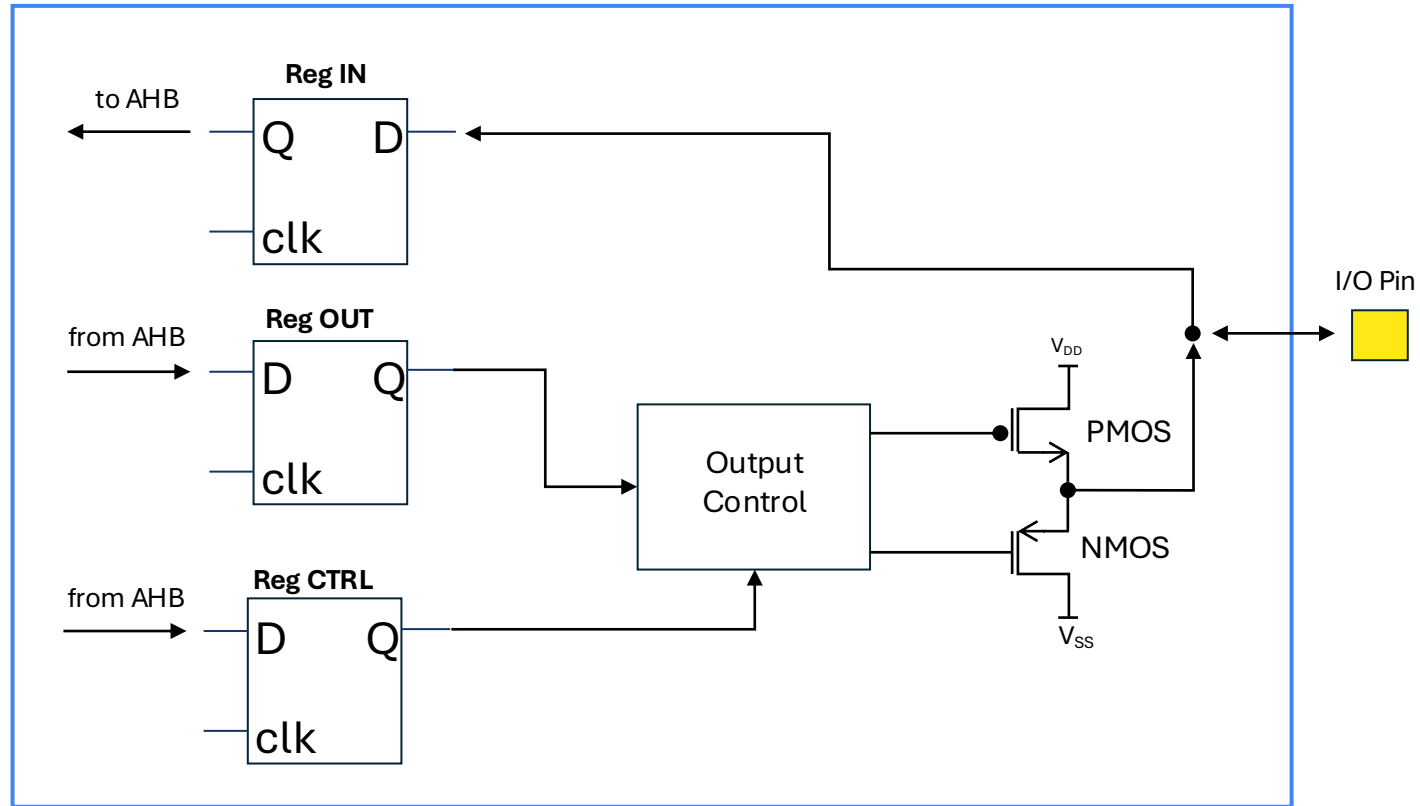
MMIO + GPIO

Getting signals on and off chip



GPIO: General Purpose Input/Output

Single GPIO Pin



GPIO Port

Address

Registers

0x033088

Port B - Input



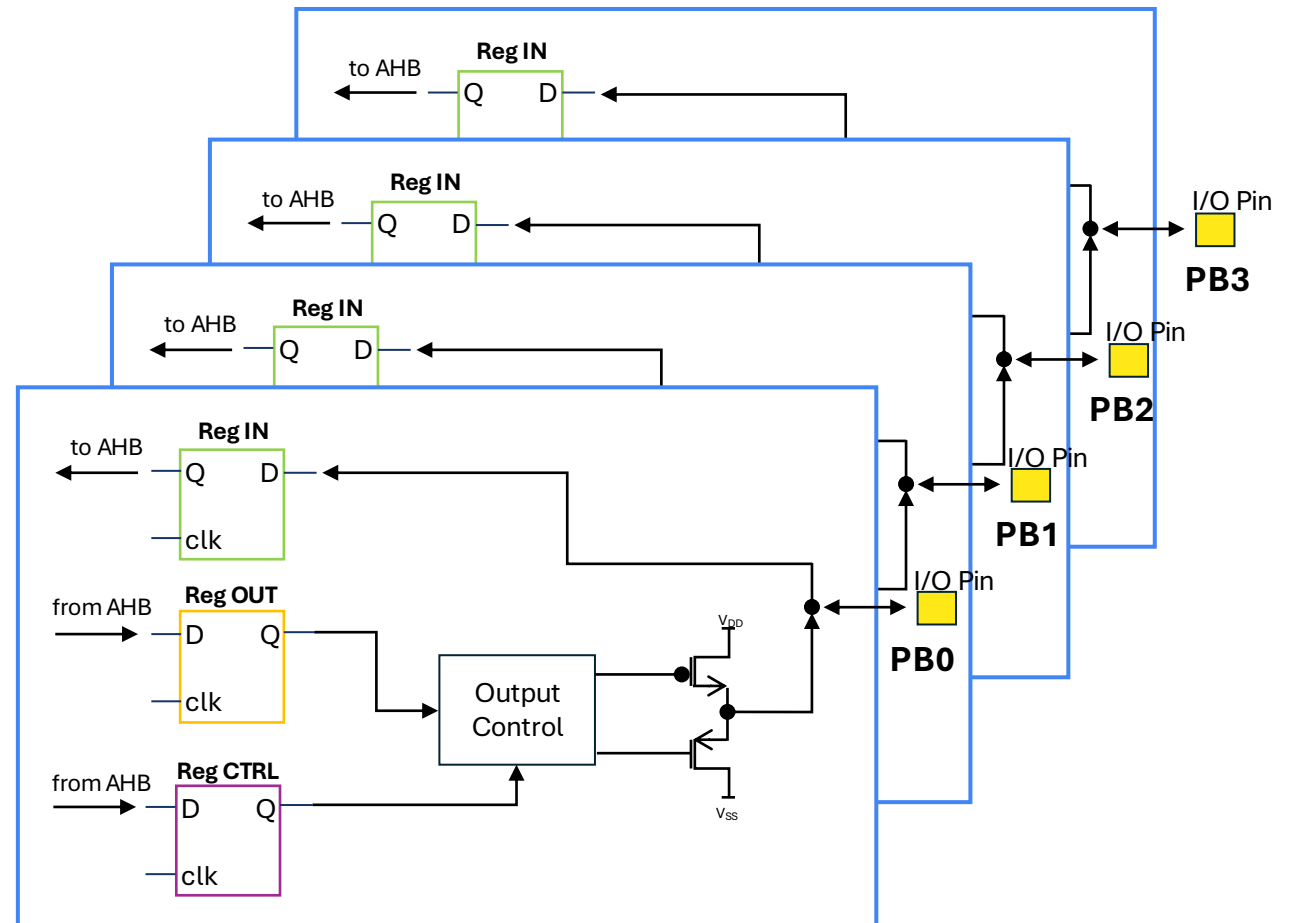
0x03308C

Port B - Output



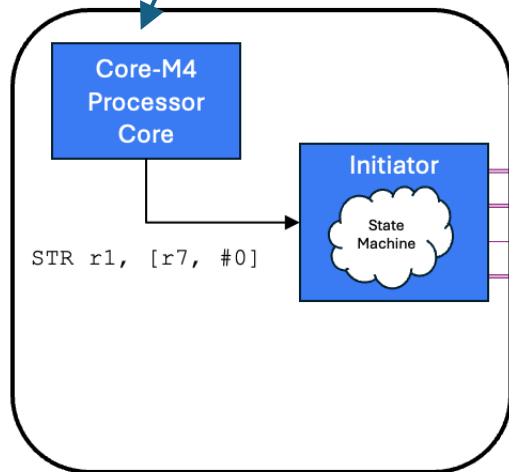
0x033080

Port B - Control



GPIO Ports

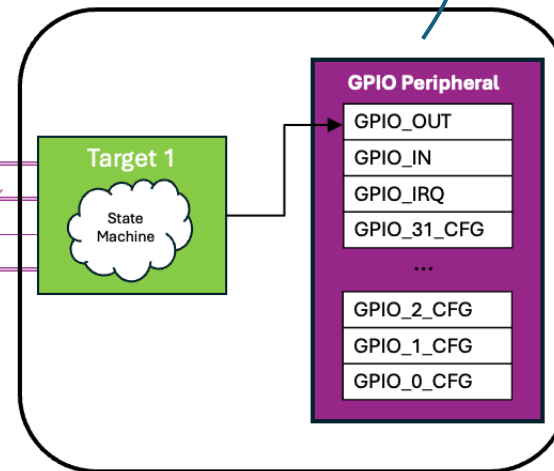
```
STR r1, [r7, #0]
```



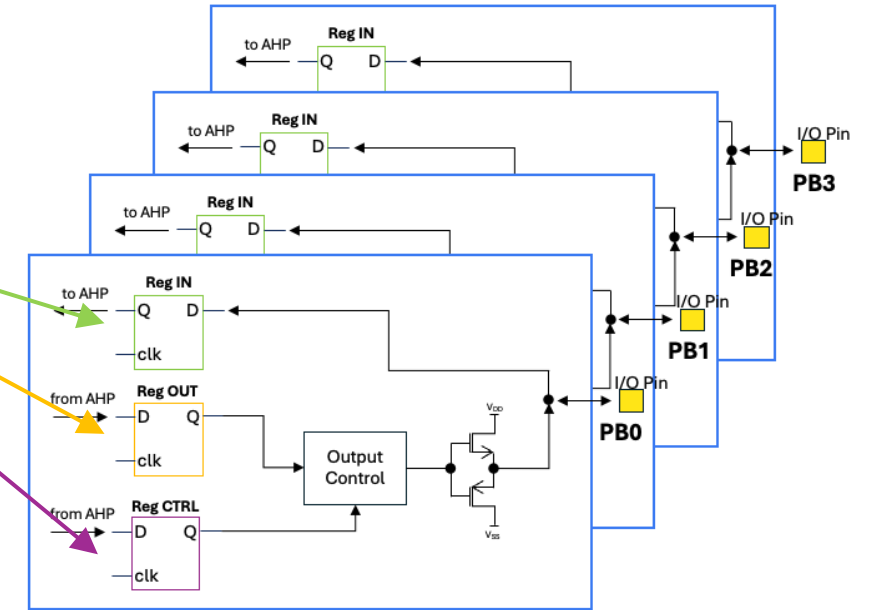
Cortex – M4 Processor Core

Address Memory
(aka peripheral registers)

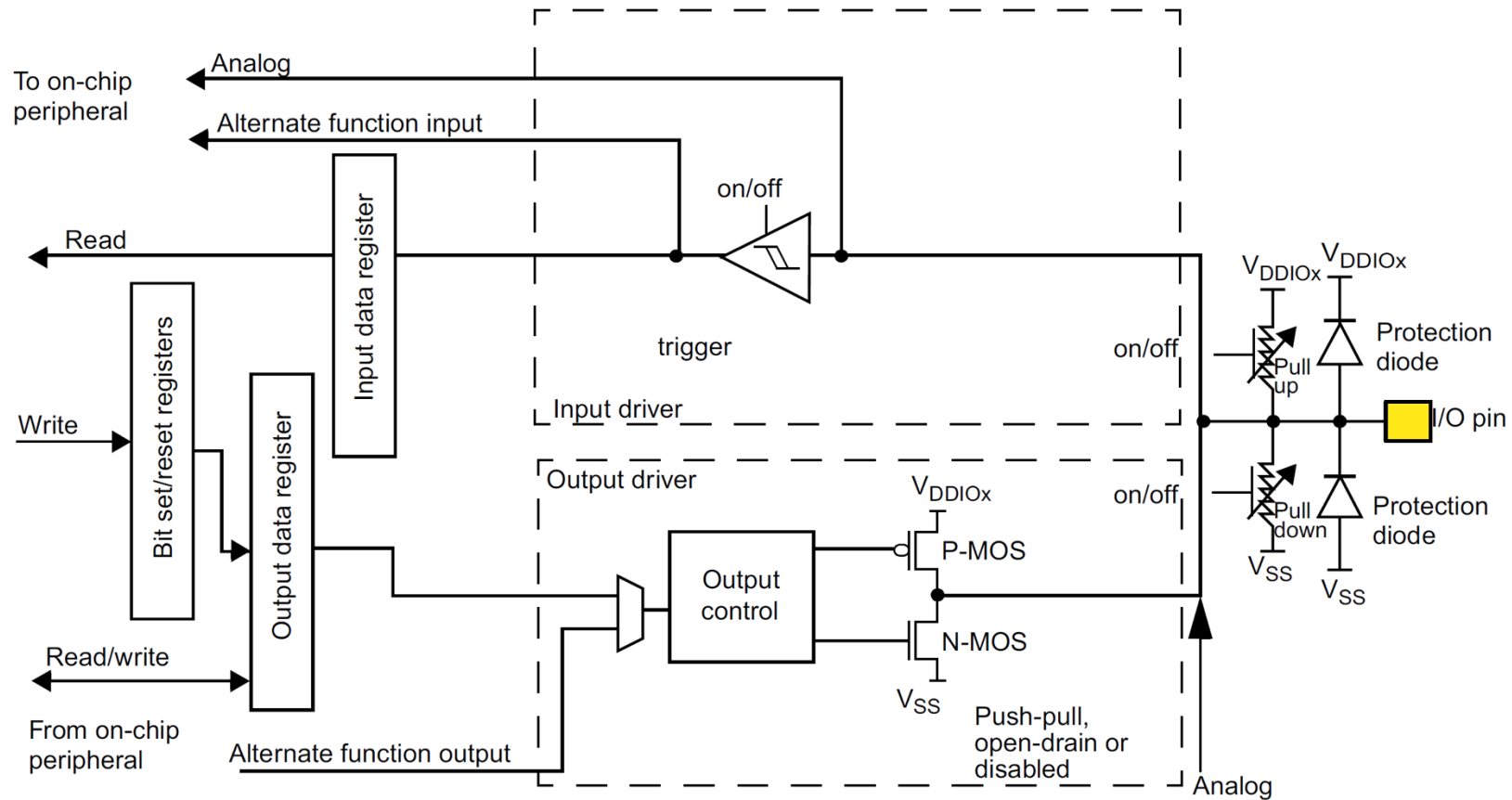
0x033088	1	0	0	0
0x03308C	0	0	1	1
0x033080	1	0	1	0



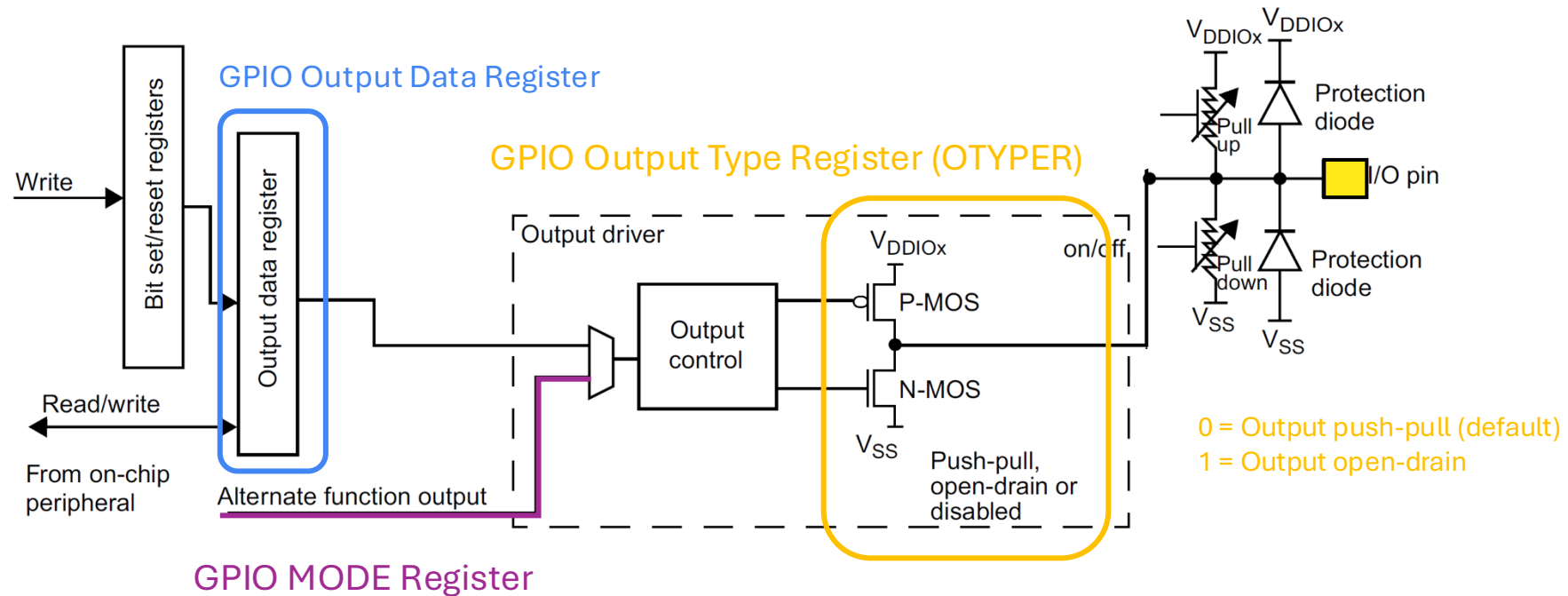
GPIO Peripheral



Basic Structure of an I/O Port (Single Bit)



Basic Structure of an I/O Port Bit : Output



Enable Clock

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	RNG EN	Res.	AESEN
													rw		rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Res.	Res.	ADCEN	OTGFS EN	Res.	Res.	Res.	Res.	GPIOH EN	GPIOG EN	GPIOF EN	GPIOE EN	GPIOD EN	GPIOC EN	GPIOB EN	GPIOA EN
		rw	rw					rw	rw	rw	rw	rw	rw	rw	rw

Bit 1 **GPIOBEN**: IO port B clock enable

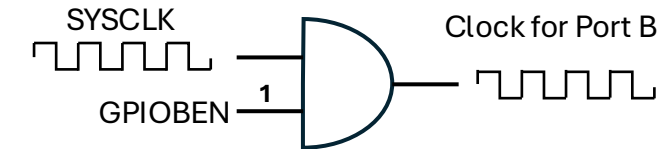
Set and cleared by software.

0: IO port B clock disabled

1: IO port B clock enabled

```
#define RCC_AHB2ENR_GPIOBEN ((uint32_t)0x00000002U)
```

```
RCC->AHB2ENR |= RCC_AHB2ENR_GPIOBEN;
```



GPIO Mode Register (MODER)

32 bits (16 pins, 2 bits per pin)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
MODE15[1:0]		MODE14[1:0]		MODE13[1:0]		MODE12[1:0]		MODE11[1:0]		MODE10[1:0]		MODE9[1:0]		MODE8[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
MODE7[1:0]		MODE6[1:0]		MODE5[1:0]		MODE4[1:0]		MODE3[1:0]		MODE2[1:0]		MODE1[1:0]		MODE0[1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
										Pin 2		Pin 1		Pin 0	

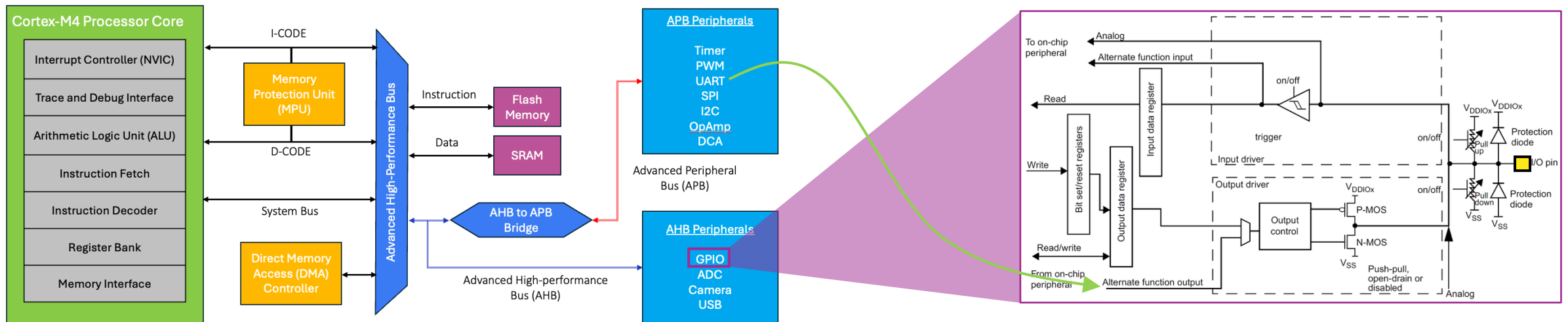
Bits 2y+1:2y **MODEy[1:0]**: Port x configuration bits (y = 0..15)
These bits are written by software to configure the I/O mode.

- 00: Input mode
- 01: General purpose output mode
- 10: Alternate function mode
- 11: Analog mode (reset state)

```
GPIOB->MODER &= ~(3UL<<4); // Clear bits 4 and 5 for Pin 2
GPIOB->MODER |= 1UL<<4; // Set bit 4, set Pin 2 as output
```

Example: Alternative Functions

Alternative functions could be UART, SPI, I2C, etc.



GPIO Output Type Register (OTYPE)

16 bits reserved, 16 data bits, 1 bit for each pin

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OT15	OT14	OT13	OT12	OT11	OT10	OT9	OT8	OT7	OT6	OT5	OT4	OT3	OT2	OT1	OT0
rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW	rW

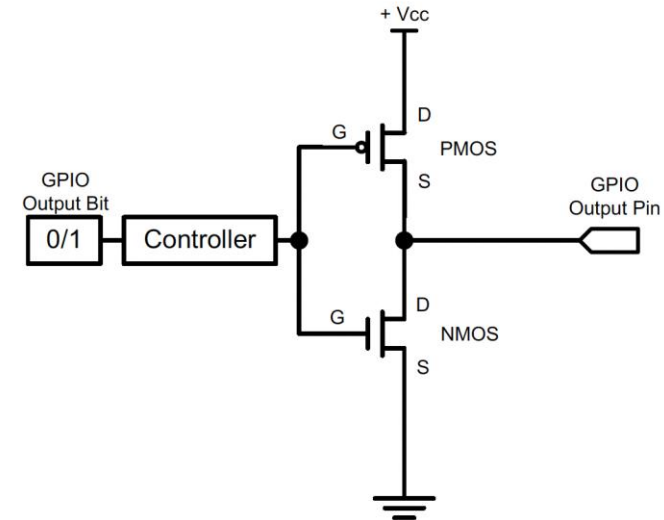
Bits 15:0 **OTy**: Port x configuration bits (y = 0..15)

These bits are written by software to configure the I/O output type.

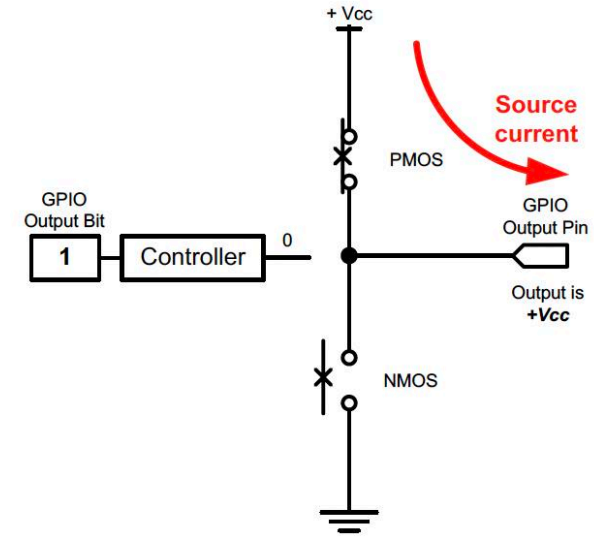
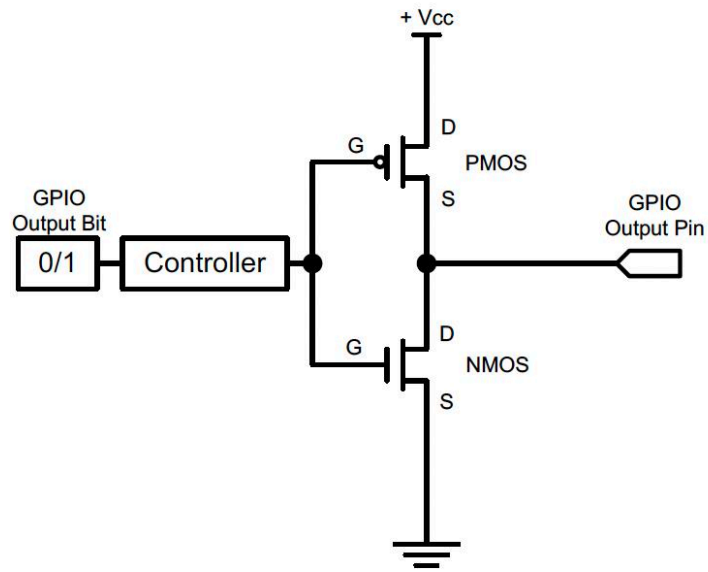
0: Output push-pull (reset state)

1: Output open-drain

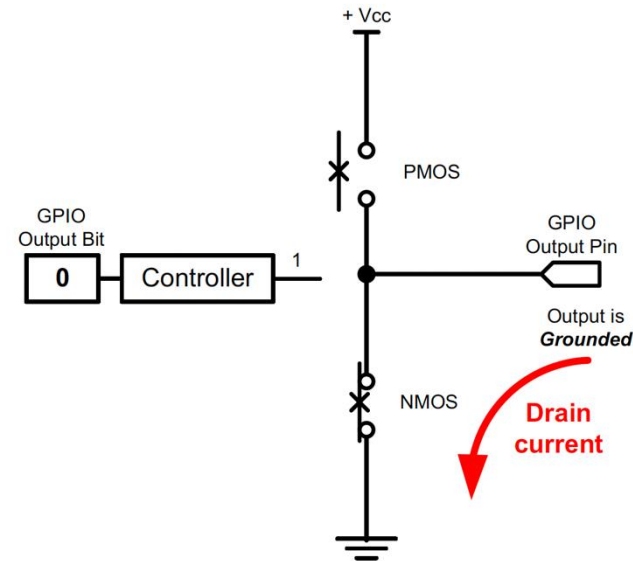
```
GPIOB->OTYPE &= ~(1UL<<2);    // Clear bit 2
```



GPIO Output: Push-pull

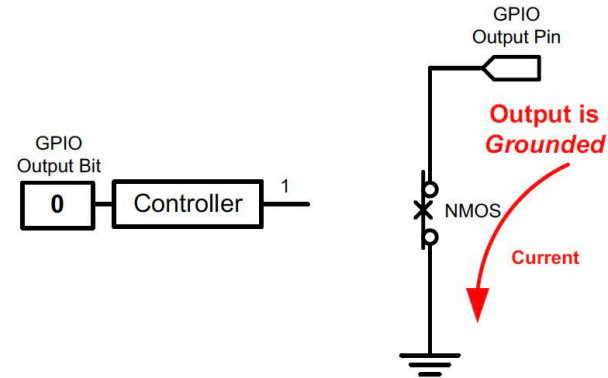
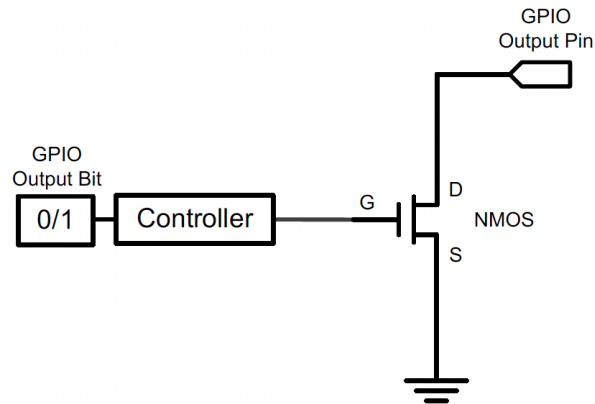


GPIO Output = 1
Source current to external circuit

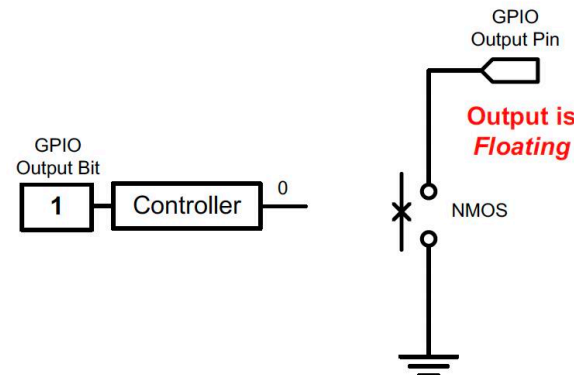


GPIO Output = 0
Drain current from external circuit

GPIO: Open Drain



GPIO Output = 0
Drain current from external circuit



GPIO Output = 1
GPIO Pin has high-impedance to external circuit

GPIO Output Speed

Configure the GPIO output speed

Output Speed:

- Speed of rising and falling
- Four speeds – Low, Medium, Fast, High

Tradeoffs:

- Higher GPIO speed increases EMI noise and power consumption
- Configure based on peripheral speed
 - Low speed for toggling LEDs
 - High speed for SPI

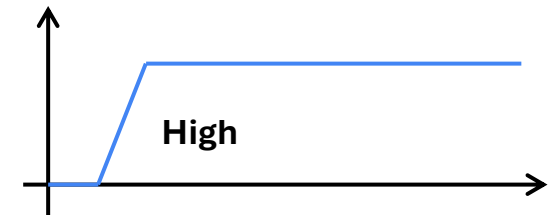
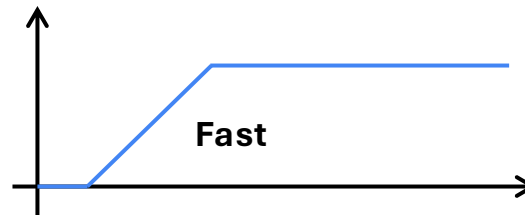
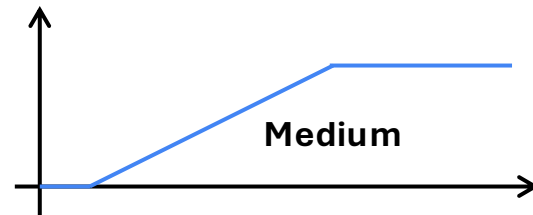
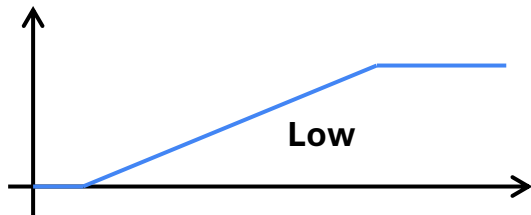
8.4.3 GPIO port output speed register (GPIOx_OSPEEDR) (x = A to I)

Address offset: 0x08

Reset value: 0x0C00 0000 (for port A)

Reset value: 0x0000 0000 (for the other ports)

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
OSPEED15 [1:0]		OSPEED14 [1:0]		OSPEED13 [1:0]		OSPEED12 [1:0]		OSPEED11 [1:0]		OSPEED10 [1:0]		OSPEED9 [1:0]		OSPEED8 [1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OSPEED7 [1:0]		OSPEED6 [1:0]		OSPEED5 [1:0]		OSPEED4 [1:0]		OSPEED3 [1:0]		OSPEED2 [1:0]		OSPEED1 [1:0]		OSPEED0 [1:0]	
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw



GPIO Output Data Register (ODR)

16 bits reserved, 16 data bits, 1 bit for each pin

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16
Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.	Res.
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
OD15	OD14	OD13	OD12	OD11	OD10	OD9	OD8	OD7	OD6	OD5	OD4	OD3	OD2	OD1	OD0
rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw	rw

```
GPIOB->ODR |= 1UL << 2;           // Set bit 2
```

Example: Set Port B, Pin 2 (PB.2) to HIGH

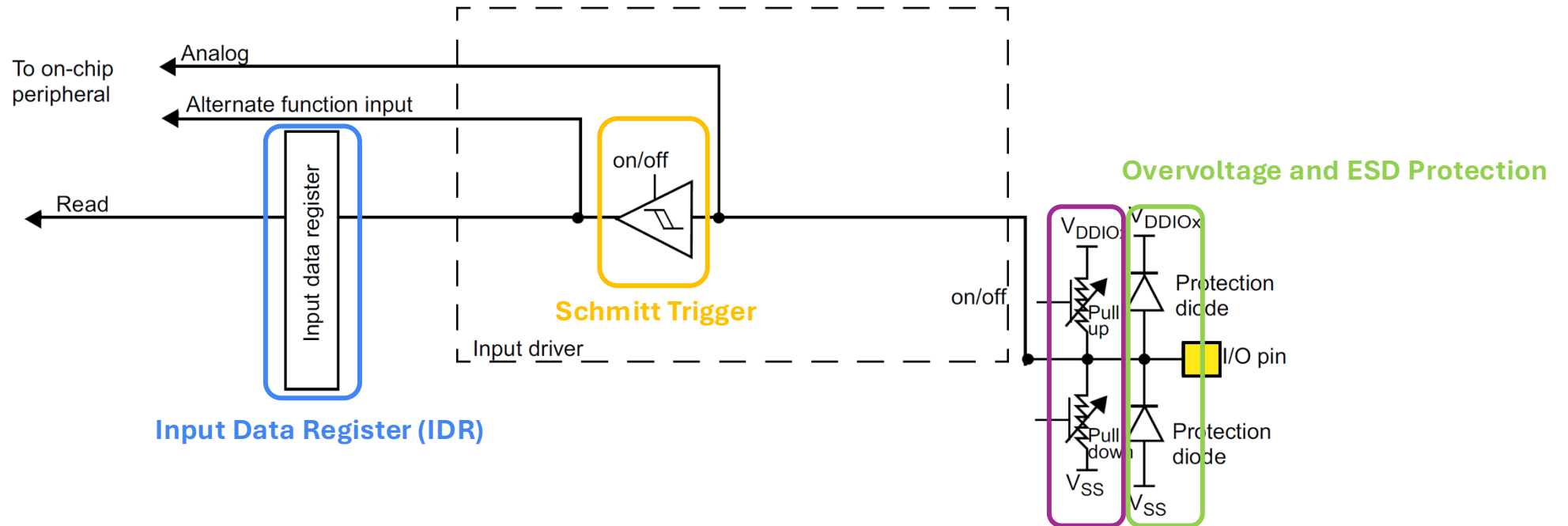
```
RCC->AHB2ENR |= RCC_AHB2ENR_GPIOBEN; // Enable clock of Port B

GPIOB->MODER  &= ~(3UL<<4);           // Clear mode bits
GPIOB->MODER  |= 1UL<<4;                // Set mode to output

GPIOB->OTYPE  &= ~(1UL<<2);           // Select push-pull output

GPIOB->ODR    |= 1UL << 2;             // Output logical "1" to drive PB.2 high
```

Basic Structure of an I/O Port Bit : Input



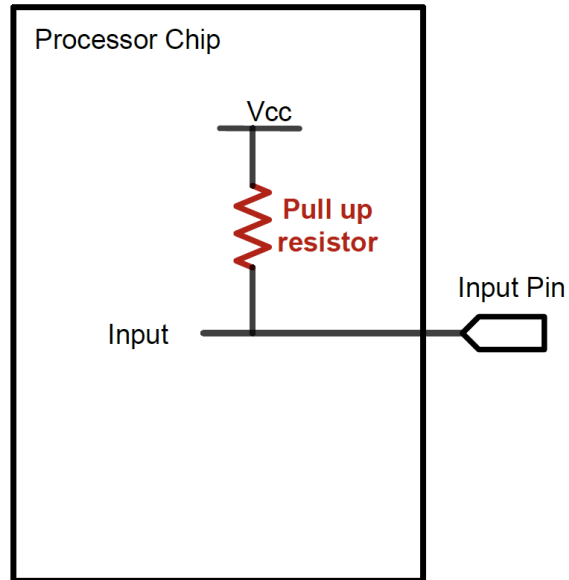
GPIO Pull-up/Pull-down Register (PUPDR)

- 00 = No pull-up, pull-down
- 10 = Pull-down
- 01 = Pull-up
- 11 = Reserved

GPIO Input: Pull up and Pull Down

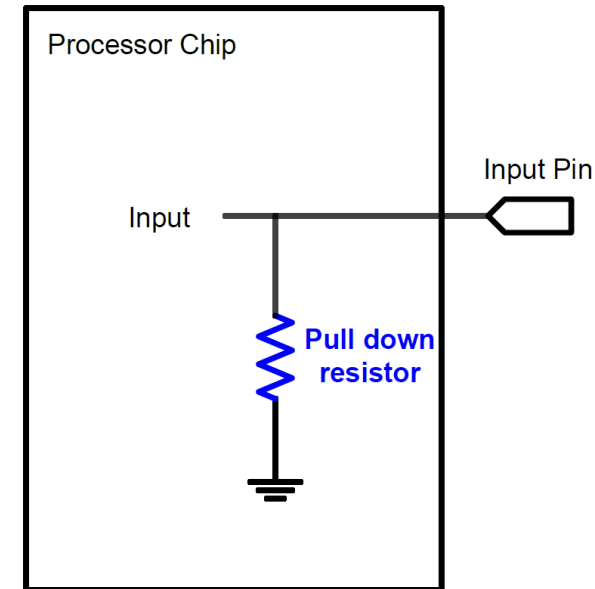
Digital input can have three states: HIGH, LOW, and High-Impedance

Pull-up



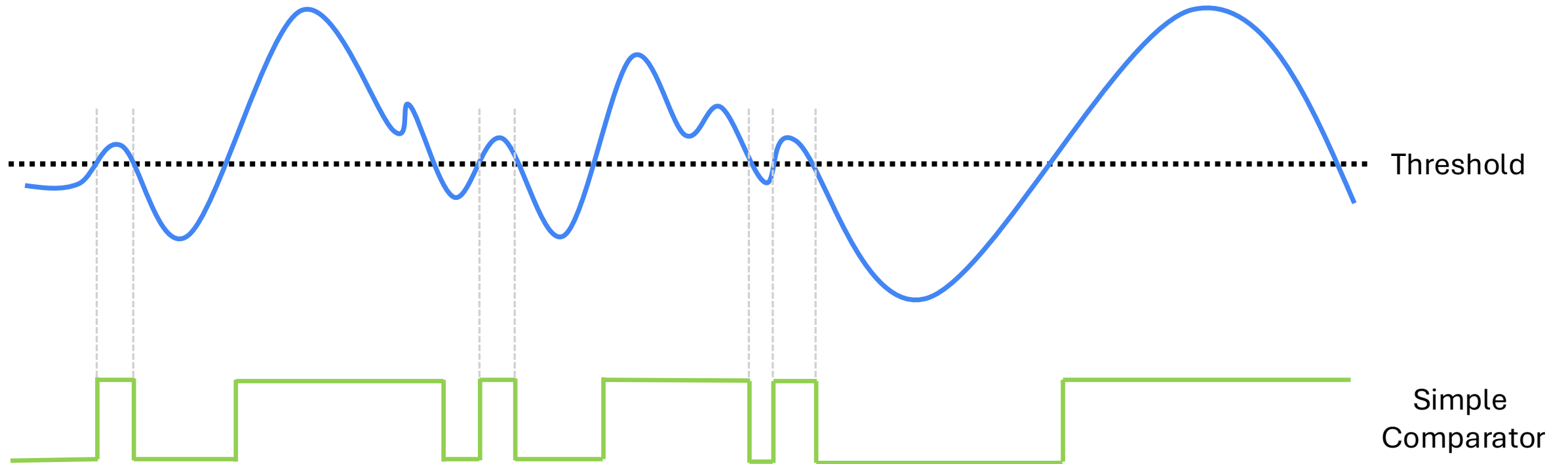
If the external input is HiZ, the input is read as HIGH

Pull-down

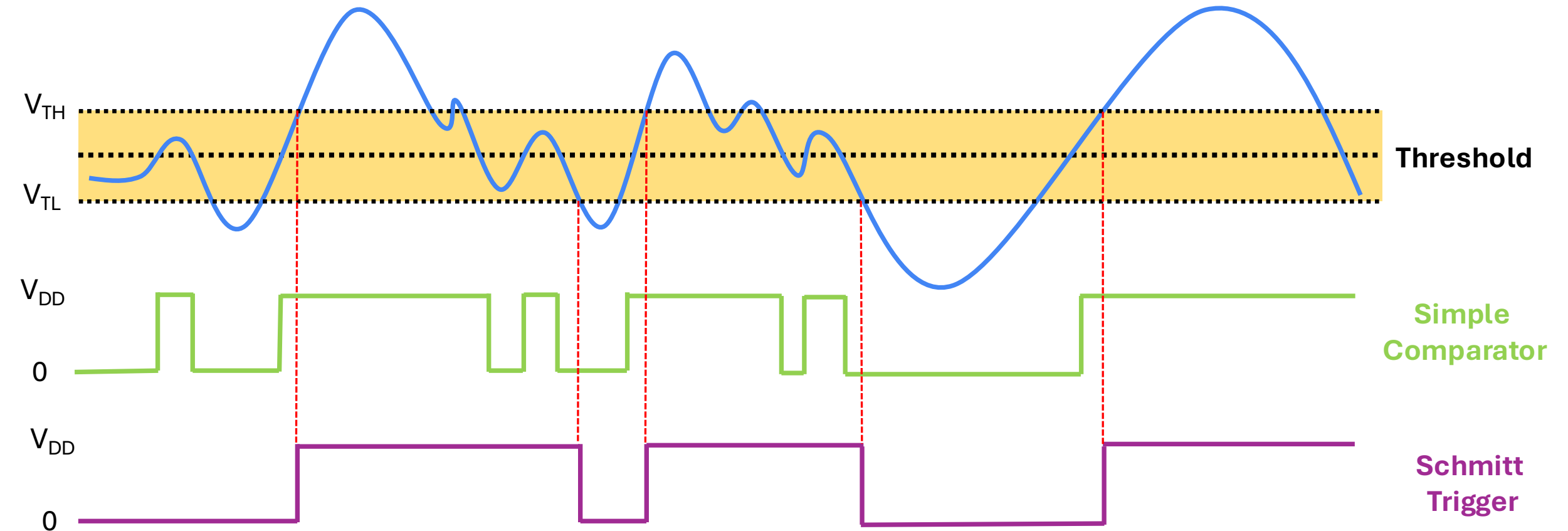


If the external input is HiZ, the input is read as LOW

Noisy Digital Signals

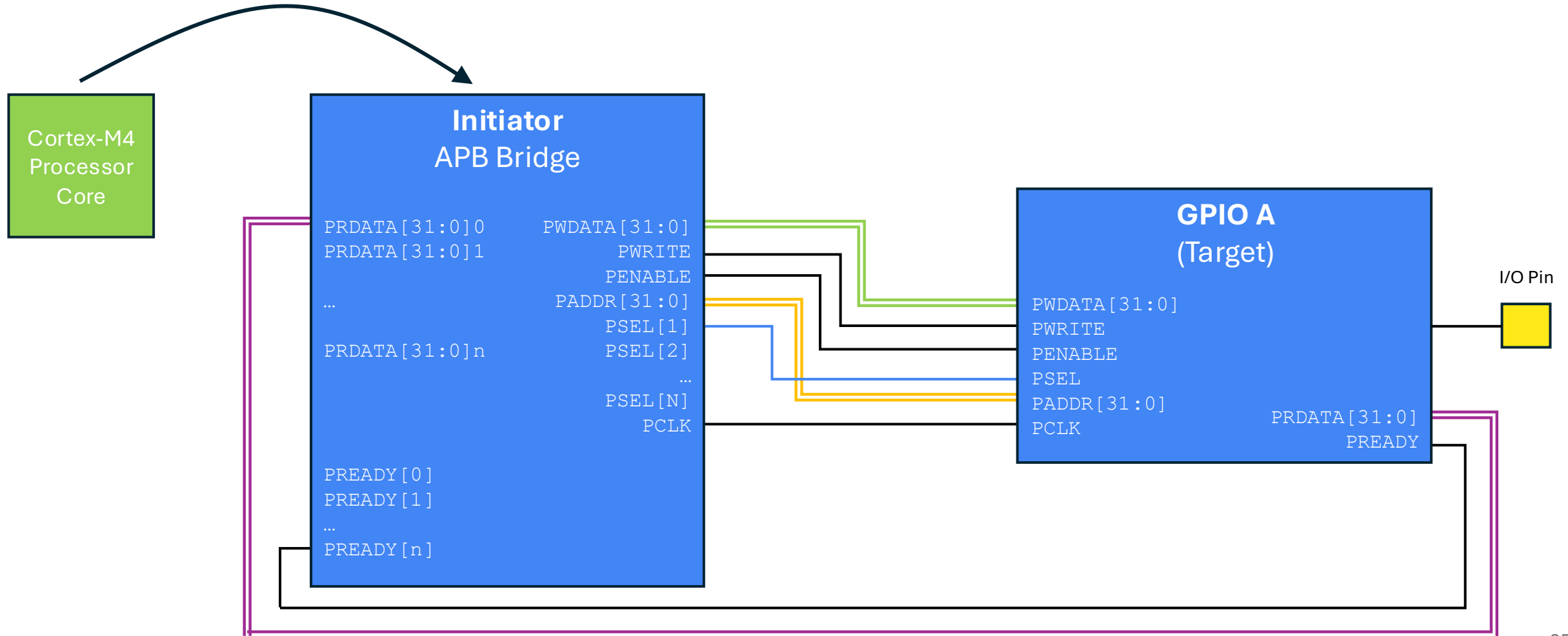


Schmitt Trigger



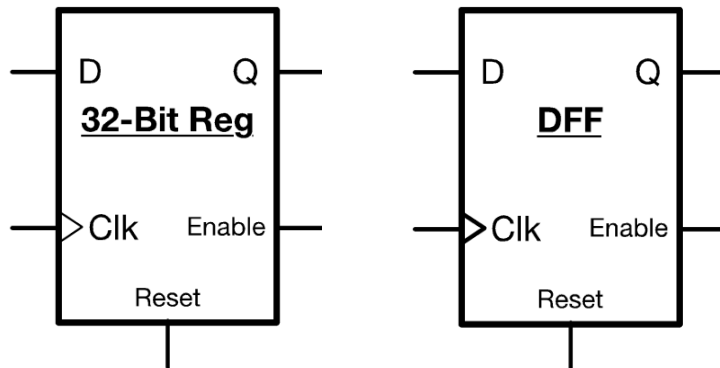
Group Exercise: Create an APB GPIO Peripheral

We are going to assume that GPIO is part of the APB rather than AHB

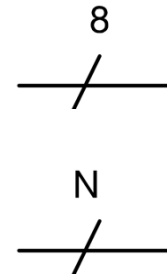


Some Standard Notation

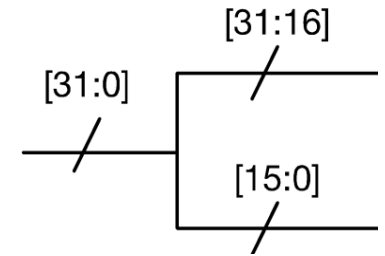
Expand DFF and Registers



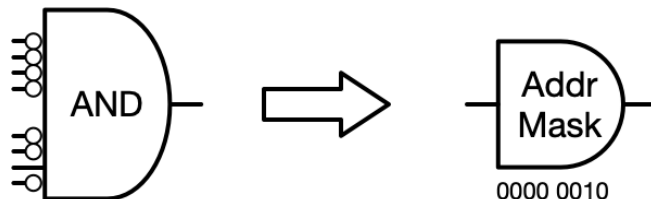
Label the number of lines in the bus



Bus concatenation and data labeling



Selecting Address Lines



VDD $\top$ = logical '1'
GND $\downarrow$ = logical '0'



Register Details

GPIO Input Register (GPIO_IN)

Address: 0x4800 0008
Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved																INPUT [15:0]															
																r	r	r	r	r	r	r	r	r	r	r	r	r	r	r	

Bits 31:15 Reserved
Bits 14:0 **INPUT** represents received in the input pin

GPIO Output Register (GPIO_OUT)

Address: 0x4800 0004
Reset value: 0x0000 0000

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved																OUTPUT [15:0]															
																r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

Bits 31:15 Reserved
Bits 14:0 **OUTPUT** represents the data that will be output on the individual pins

GPIO Mode Register (GPIO_MODE)

Address: 0x4800 0000
Reset value: 0xFFFF FFFF

31	30	29	28	27	26	25	24	23	22	21	20	19	18	17	16	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Reserved																Mode [15:0]															
																r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w	r/w

Bits 15:0 Each pin can be configured configured as either an input or an output. Setting MODE has the following functions:
00: General purpose output
01: Input

Group Exercise:

- Wire this APB peripheral to the APB to enable full GPIO read/write functionality
- Assume everything is ready in one clock cycle
- Assume *GPIO_IN* is updating every clock cycle

PWDATA[31:0] →

PRDATA[31:0] ←

PWRITE →

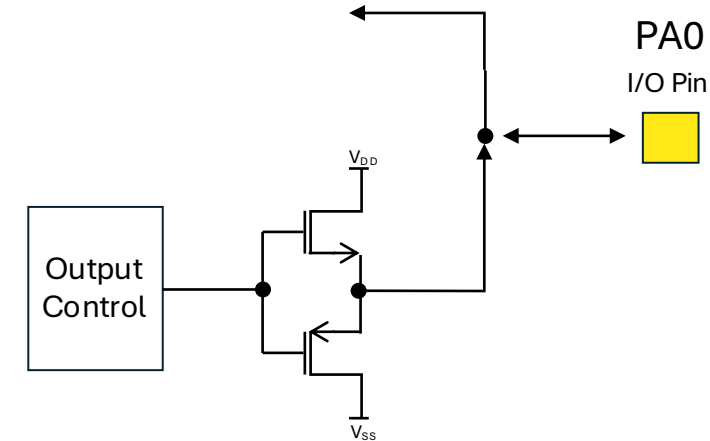
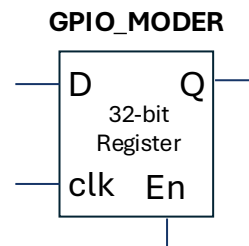
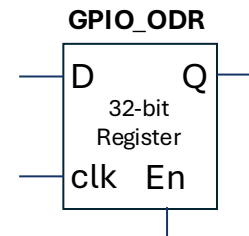
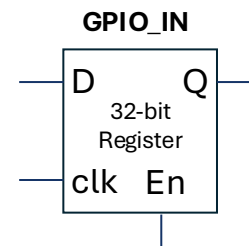
PENABLE →

PSEL →

PADDR[31:0] →

PCLK →

PREADY←



Recap

GPIO – General Purpose Input / Output

- On chip hardware that controls the configurations of “pins”
- Configure to output or capture input

MMIO – Memory Mapped Input / Output

- A means to interface with peripherals (including GPIO) from the processor core using Loads and Stores
- Use “memory mapped” registers to configure GPIO pins
- Use “memory mapped” registers to read and write from pin

Acknowledgements

- These lecture slides were adapted from the University of Michigan's EECS 373 course (many thanks to Alanson Sample!)
- These lecture slides also leverage materials from the following books:
 - Embedded Systems with ARM Cortex-M Microcontrollers in Assembly Language and C
 - Yifeng Zhu
 - The Definitive Guide to ARM Cortex – M3 and Cortex – M4 Processors
 - Joseph Yiu
 - Introduction to ARM Cortex – M Microcontrollers, Embedded Systems
 - Jonathan W. Valvano